

HỌC VIỆN HÀNG KHÔNG VIỆT NAM

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

ĐIỀU KHIỂN TRUY XUẤT ĐỒNG THỜI TRONG MYSQL/INNODB VÀ XÂY DỰNG ỨNG DỤNG MINH HỌA ĐẶT VÉ MÁY BAY

Giảng viên hướng dẫn:

Sinh viên/ Nhóm sinh viên thực hiện: Nhóm **01**

Mã số sinh viên:

Lớp: **0101000xxxxx**

TP.Hồ Chí Minh, tháng 10/2026

HỌC VIỆN HÀNG KHÔNG VIỆT NAM

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU

ĐIỀU KHIỂN TRUY XUẤT ĐỒNG THỜI TRONG MYSQL/INNODB VÀ XÂY DỰNG ỨNG DỤNG MINH HỌA ĐẶT VÉ MÁY BAY

Giảng viên hướng dẫn:

Sinh viên/ Nhóm sinh viên thực hiện: Nhóm **01**

Mã số sinh viên:

Lớp: **0101000xxxxx**

Thành phố Hồ Chí Minh, tháng 10/2026

Danh sách Nhóm:

STT	Họ và tên	MSSV	Lớp	Ghi chú
1	Đàm Thanh Vũ	...	...	
2	...			
3	...			
4	...			
5	...			

Cán bộ chấm thi 1 <i>(ký và ghi rõ họ tên)</i>	Cán bộ chấm thi 2 <i>(ký và ghi rõ họ tên)</i>
Cán bộ chấm thi phúc khảo 1 <i>(ký và ghi rõ họ tên)</i>	Cán bộ chấm thi phúc khảo 2 <i>(ký và ghi rõ họ tên)</i>

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT

Ký hiệu, chữ viết tắt	Chữ viết đầy đủ
2PL	Two-Phase Locking – giao thức khóa hai pha
ACID	Atomicity, Consistency, Isolation, Durability – tính nguyên tử, nhất quán, cô lập, bền vững
API	Application Programming Interface – giao diện lập trình ứng dụng
CSDL	Cơ sở dữ liệu
FK	Foreign Key – khóa ngoại
HQTCSDL	Hệ quản trị cơ sở dữ liệu (Database Management System – DBMS)
IS / IX	Intention Shared / Intention Exclusive lock – khóa ý định đọc / khóa ý định ghi
MVCC	Multi-Version Concurrency Control – điều khiển đồng thời đa phiên bản
PK	Primary Key – khóa chính
RU / RC / RR	READ UNCOMMITTED / READ COMMITTED / REPEATABLE READ – các mức cô lập
S / X	Shared / Exclusive lock – khóa chia sẻ (khóa đọc) / khóa độc quyền (khóa ghi)
SQL	Structured Query Language – ngôn ngữ truy vấn có cấu trúc
UI	User Interface – giao diện người dùng
WAL	Write-Ahead Logging – ghi nhật ký trước khi ghi dữ liệu

MỤC LỤC

Trang

DANH MỤC CÁC KÝ HIỆU, CHỮ VIẾT TẮT	iv
DANH MỤC CÁC BẢNG, HÌNH VẼ, ĐỒ THỊ	viii
MỞ ĐẦU	1
CHƯƠNG 1. GIỚI THIỆU	2
1.1. Lý do chọn đề tài	2
1.2. Mục tiêu đề tài	2
1.3. Phạm vi đề tài	3
1.4. Đối tượng nghiên cứu	4
1.5. Phương pháp nghiên cứu	4
1.6. Bố cục đề tài	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	6
2.1. Giao tác và lịch giao tác	6
2.1.1. Khái niệm giao tác	6
2.1.2. Các tính chất ACID	6
2.1.3. Các trạng thái của giao tác	7
2.1.4. Lịch giao tác và tính khả tuần tự	7
2.2. Các vấn đề khi truy xuất đồng thời	8
2.2.1. Mất cập nhật (Lost update)	8
2.2.2. Đọc dữ liệu rác (Dirty read)	9
2.2.3. Đọc không lặp lại (Non-repeatable read)	9
2.2.4. Bóng ma (Phantom read)	9
2.2.5. Khóa chết (Deadlock)	9
2.3. Các kỹ thuật điều khiển truy xuất đồng thời	10
2.3.1. Kỹ thuật khóa	10
2.3.2. Giao thức khóa hai pha (2PL)	10
2.3.3. Điều khiển đồng thời đa phiên bản (MVCC)	11
2.3.4. Khóa bi quan và khóa lạc quan	11
2.3.5. Xử lý khóa chết	12

2.4. Mức cô lập giao tác theo chuẩn SQL	12
2.5. Hệ quản trị CSDL MySQL và storage engine InnoDB	12
2.5.1. Lịch sử hình thành và kiến trúc.....	12
2.5.2. Mô hình giao tác của InnoDB.....	13
2.5.3. Các loại khóa trong InnoDB.....	14
2.5.4. Cơ chế đa phiên bản (MVCC) trong InnoDB	15
2.5.5. Phát hiện và xử lý khóa chết trong InnoDB.....	16
2.5.6. Giám sát khóa qua performance_schema.....	16
2.6. Công nghệ xây dựng ứng dụng minh họa	17
CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG VÀ XÂY DỰNG SẢN PHẨM.....	18
3.1. Phân tích hệ thống	18
3.1.1. Yêu cầu chức năng	18
3.1.2. Yêu cầu phi chức năng	19
3.1.3. Sơ đồ use case	19
3.1.4. Kiến trúc hệ thống.....	20
3.1.5. Thiết kế cơ sở dữ liệu	21
3.1.6. Thiết kế năm cặp kịch bản xung đột.....	22
3.1.7. Bộ điều phối hai phiên	23
3.1.8. Thiết kế bốn cách đặt vé đồng thời	24
3.1.9. Giám sát khóa và các vấn đề gặp phải	25
3.2. Xây dựng giao diện sản phẩm	26
3.2.1. Giao diện tổng quan và điều hướng.....	26
3.2.2. Chức năng năm xung đột: sai và đúng	28
3.2.3. Xung đột 1: Mất cập nhật (Lost update)	29
3.2.4. Xung đột 2: Đọc dữ liệu rác (Dirty read)	33
3.2.5. Xung đột 3: Đọc không lặp lại (Non-repeatable read).....	36
3.2.6. Xung đột 4: Bóng ma (Phantom read)	39
3.2.7. Xung đột 5: Khóa chết (Deadlock)	42
3.2.8. Chức năng mô phỏng nhiều khách đặt vé	46
3.2.9. Chức năng ma trận mức cô lập.....	51

3.2.10. Chức năng so sánh hiệu năng	54
3.3. Kiểm thử và triển khai.....	57
3.4. Đánh giá và những phát hiện về MySQL/InnoDB.....	58
KẾT LUẬN	59
DANH MỤC TÀI LIỆU THAM KHẢO	61
PHỤ LỤC	63
Phụ lục A. Hướng dẫn cài đặt và chạy	63
Phụ lục B. Lược đồ cơ sở dữ liệu (schema.sql)	63
Phụ lục C. Cấu trúc mã nguồn.....	64

DANH MỤC CÁC BẢNG, HÌNH VẼ, ĐỒ THỊ

Danh mục bảng

Bảng 2.1. Bốn tính chất ACID của giao tác.....	7
Bảng 2.2. Lịch giao tác gây mất cập nhật: $r_1(X) \ r_2(X) \ w_1(X) \ c_1 \ w_2(X) \ c_2$	8
Bảng 2.3. Ma trận tương thích khóa mức bảng của InnoDB	10
Bảng 2.4. So sánh khóa bị quan và khóa lạc quan	11
Bảng 2.5. Các mức cô lập và hiện tượng được phép xảy ra theo chuẩn SQL	12
Bảng 2.6. Các công nghệ sử dụng.....	17
Bảng 3.1. Yêu cầu chức năng	18
Bảng 3.2. Yêu cầu phi chức năng	19
Bảng 3.3. Các API của ứng dụng.....	20
Bảng 3.4. Mô tả các cột của ba bảng	21
Bảng 3.5. Thiết kế năm cặp kịch bản sai / đúng	22
Bảng 3.6. Kết quả đo được của năm cặp kịch bản.....	46
Bảng 3.7. Kết quả một lần chạy với 10 khách, 5 ghế, cân nhắc 100 ms	51
Bảng 3.8. Kết quả ma trận đo trên MySQL (* = khác lý thuyết chuẩn SQL)	52
Bảng 3.9. Kết quả trung bình của bốn cách xử lý.....	55
Bảng 3.10. Khảo sát chịu tải (cân nhắc 50 ms; số ghế bằng một nửa số khách)	56
Bảng 3.11. Các nhóm kiểm thử tự động	57

Danh mục hình vẽ, đồ thị

Hình 2.1. Sơ đồ trạng thái của một giao tác.....	7
Hình 2.2. Đồ thị chờ của hai giao tác bị khóa chết.....	10
Hình 2.3. Kiến trúc phân tầng của MySQL và các thành phần chính của InnoDB.....	13
Hình 2.4. Record lock, gap lock và next-key lock trên chỉ mục ma_chuyen.....	15
Hình 2.5. MVCC: cùng một dòng, các cách đọc khác nhau thấy các phiên bản khác nhau	16
Hình 3.1. Sơ đồ use case của ứng dụng minh họa	19
Hình 3.2. Kiến trúc tổng thể của ứng dụng minh họa.....	20
Hình 3.3. Lược đồ cơ sở dữ liệu demo_dongthoi.....	21

Hình 3.4. Lưu đồ bộ điều phối hai phiên	24
Hình 3.5. Giao diện tổng quan của ứng dụng	27
Hình 3.6. Khi thanh trên đang hiện, một phần lịch giao tác bị che	28
Hình 3.7. Sau khi bấm “Ấn thanh trên”, lịch giao tác và bảng CSDL hiện đầy đủ.....	28
Hình 3.8. Danh sách năm xung đột.....	29
Hình 3.9. Chọn kịch bản, hướng dẫn đọc màn hình và thanh điều khiển phát lại.....	29
Hình 3.10. Mất cập nhật – kịch bản SAI tại bước then chốt	30
Hình 3.11. Mất cập nhật – kịch bản ĐÚNG tại bước then chốt	32
Hình 3.12. Mất cập nhật – so sánh hai kịch bản	33
Hình 3.13. Đọc dữ liệu rác – kịch bản SAI tại bước then chốt.....	34
Hình 3.14. Đọc dữ liệu rác – kịch bản ĐÚNG tại bước then chốt	35
Hình 3.15. Đọc dữ liệu rác – so sánh hai kịch bản	36
Hình 3.16. Đọc không lặp lại – kịch bản SAI tại bước then chốt.....	37
Hình 3.17. Đọc không lặp lại – kịch bản ĐÚNG tại bước then chốt.....	38
Hình 3.18. Đọc không lặp lại – so sánh hai kịch bản	39
Hình 3.19. Bóng ma – kịch bản SAI tại bước then chốt.....	40
Hình 3.20. Bóng ma – kịch bản ĐÚNG tại bước then chốt.....	41
Hình 3.21. Bóng ma – so sánh hai kịch bản	42
Hình 3.22. Khóa chết – kịch bản SAI tại bước then chốt	43
Hình 3.23. Khóa chết – kịch bản ĐÚNG tại bước then chốt	45
Hình 3.24. Khóa chết – so sánh hai kịch bản.....	46
Hình 3.25. Cấu hình mô phỏng nhiều khách đặt vé.....	47
Hình 3.26. Kết quả khi không dùng khóa: bán vượt.....	47
Hình 3.27. Diễn biến bên trong CSDL khi không dùng khóa	48
Hình 3.28. Sơ đồ khoang ghế: các ghế bị bán trùng	48
Hình 3.29. Diễn biến khi dùng khóa bị quan: các giao tác xếp hàng	49
Hình 3.30. Các khóa InnoDB lấy mẫu trong lúc chạy khóa bị quan	49
Hình 3.31. Diễn biến khi dùng khóa lạc quan: nhiều lần thử lại	50
Hình 3.32. Kết quả khi dùng UPDATE nguyên tử	50
Hình 3.33. Diễn biến kịch bản deadlock: InnoDB hủy giao tác nạn nhân	50

Hình 3.34. Nhật ký câu lệnh SQL của kịch bản deadlock	51
Hình 3.35. Ma trận 6 hiện tượng $\times$ 4 mức cô lập đo trên MySQL	52
Hình 3.36. Chi tiết thí nghiệm bóng ma có khóa ở REPEATABLE READ: B bị gap lock chặn.....	54
Hình 3.37. So sánh bốn cách xử lý tranh chấp	55
Hình 3.38. Khảo sát chịu tải khi số khách tăng dần	56

MỞ ĐẦU

Trong các hệ thống thông tin hiện đại, cơ sở dữ liệu hiếm khi chỉ phục vụ một người dùng tại một thời điểm. Một hệ thống đặt vé máy bay trực tuyến có thể nhận hàng trăm yêu cầu giữ chỗ trong cùng một giây cho cùng một chuyến bay; một sàn thương mại điện tử trong đợt khuyến mãi có hàng nghìn khách cùng bấm “Mua ngay” cho một số lượng hàng có hạn. Khi nhiều giao tác cùng đọc và ghi lên cùng một dữ liệu, nếu hệ quản trị cơ sở dữ liệu không điều phối tốt thì dữ liệu sẽ sai lệch: vé bị bán vượt số ghế, số dư bị trừ sai, báo cáo đọc phải số liệu chưa từng tồn tại chính thức.

Điều khiển truy xuất đồng thời (concurrency control) là thành phần của hệ quản trị cơ sở dữ liệu có nhiệm vụ cho phép nhiều giao tác chạy xen kẽ để tận dụng tài nguyên, nhưng vẫn bảo đảm kết quả như thể chúng chạy lần lượt. Đây là một trong những nội dung trọng tâm của học phần Hệ quản trị cơ sở dữ liệu, gắn liền với các khái niệm giao tác, lịch giao tác, khóa, đa phiên bản và mức cô lập.

Tiểu luận này nghiên cứu cơ chế điều khiển truy xuất đồng thời của **MySQL 9.6.0** với storage engine **InnoDB** – hệ quản trị cơ sở dữ liệu mã nguồn mở được dùng rất rộng rãi. Bên cạnh phần lý thuyết, nhóm xây dựng một **ứng dụng minh họa** theo nghiệp vụ đặt vé máy bay, trong đó mọi kịch bản đều được chạy thật trên MySQL: năm xung đột kinh điển (mất cập nhật, đọc dữ liệu rác, đọc không lặp lại, bóng ma, khóa chết) được tái hiện dưới dạng lịch giao tác có hoạt hình, kèm kịch bản sai và kịch bản đúng chỉ khác nhau đúng một điểm; nhiều khách hàng đặt vé đồng thời với bốn cách xử lý tranh chấp; ma trận sáu hiện tượng trên bốn mức cô lập được đối chiếu với lý thuyết chuẩn SQL. Ứng dụng đọc trực tiếp bảng khóa của InnoDB, nhờ đó người xem thấy được tận mắt khóa dòng, khóa khoảng trống và vòng chờ gây ra khóa chết.

Kết quả thực nghiệm cho thấy một số điểm MySQL/InnoDB khác với lý thuyết chuẩn SQL – chẳng hạn mức REPEATABLE READ của MySQL chặn được hiện tượng bóng ma nhưng lại không chặn được mất cập nhật. Những phát hiện này được trình bày, giải thích và kiểm chứng bằng số liệu trong các chương sau.

CHƯƠNG 1. GIỚI THIỆU

1.1. Lý do chọn đề tài

Các ứng dụng trực tuyến ngày nay đều là hệ thống nhiều người dùng. Ở những nghiệp vụ có tài nguyên hữu hạn như bán vé, giữ chỗ, quản lý tồn kho hay chuyển tiền, nhiều yêu cầu thường tranh nhau cùng một dòng dữ liệu trong những khoảnh khắc rất ngắn. Một lỗi điều khiển đồng thời không gây ra thông báo lỗi nào, ứng dụng vẫn chạy bình thường, nhưng dữ liệu đã sai: hai khách cùng nhận được một ghế, hoặc số ghế còn lại không khớp với số vé đã bán. Những lỗi như vậy rất khó phát hiện khi kiểm thử bằng tay vì chúng chỉ xuất hiện với một thứ tự xen kẽ nhất định của các câu lệnh.

Trong khi đó, nhiều lập trình viên sử dụng hệ quản trị cơ sở dữ liệu với giả định rằng “đã có giao tác thì dữ liệu sẽ đúng”, hoặc hiểu các mức cô lập theo bảng lý thuyết của chuẩn SQL mà không biết mỗi hệ quản trị cài đặt chúng theo cách riêng. Với MySQL/InnoDB, mức mặc định REPEATABLE READ không ngăn được mất cập nhật – điều trái với trực giác của nhiều người. Ở phía học tập, lịch giao tác thường chỉ được vẽ trên bảng, người học khó hình dung khi nào một giao tác bị chặn, khóa nào đang được giữ, và vì sao hệ quản trị lại chọn hủy một giao tác khi xảy ra khóa chết.

Từ thực tế đó, nhóm chọn đề tài **điều khiển truy xuất đồng thời trong MySQL/InnoDB** kết hợp xây dựng một ứng dụng minh họa. Nếu giải quyết được, đề tài vừa giúp người học hiểu sâu lý thuyết giao tác thông qua quan sát hành vi thật của một hệ quản trị phổ biến, vừa giúp người phát triển ứng dụng lựa chọn đúng kỹ thuật (khóa bị quan, khóa lạc quan, câu lệnh nguyên tử, mức cô lập phù hợp) khi xây dựng các chức năng có tranh chấp dữ liệu. Cách tiếp cận này cũng áp dụng được cho các nghiệp vụ tương tự như bán hàng khuyến mãi, ngân hàng hay quản lý kho.

1.2. Mục tiêu đề tài

Hệ thống được xây dựng là một **ứng dụng web minh họa** dành cho sinh viên và giảng viên học phần Hệ quản trị cơ sở dữ liệu. Ứng dụng giúp người dùng quan sát trực quan các xung đột khi nhiều giao tác truy xuất đồng thời và cách InnoDB xử lý chúng, với dữ liệu, câu lệnh, kết quả và bảng khóa đều lấy trực tiếp từ MySQL chứ không mô phỏng.

Kết quả cần đạt được gồm:

- Về lý thuyết: hệ thống hóa các khái niệm giao tác, tính chất ACID, lịch giao tác và tính khả tuần tự; các hiện tượng bất thường khi truy xuất đồng thời; kỹ thuật khóa, giao thức khóa hai pha, điều khiển đa phiên bản (MVCC); các mức cô lập theo chuẩn SQL và cách InnoDB cài đặt chúng.
- Về ứng dụng: tái hiện được 5 xung đột (mất cập nhật, đọc dữ liệu rác, đọc không lặp lại, bóng ma, khóa chết), mỗi xung đột có một kịch bản SAI và một kịch bản ĐÚNG chạy trên cùng dữ liệu, cùng thứ tự xen kẽ; hiển thị lịch giao tác dạng hoạt hình, dữ liệu đã commit / chưa commit và bảng khóa tại từng bước.
- Về thực nghiệm: mô phỏng nhiều khách đặt vé đồng thời với 4 cách xử lý (không khóa, khóa bi quan, khóa lạc quan, UPDATE nguyên tử), đo thời gian, số vé bán vượt, số lần chờ khóa, số lần thử lại; lập ma trận 6 hiện tượng $\times$ 4 mức cô lập và đối chiếu với lý thuyết chuẩn SQL.
- Về công nghệ: đóng gói toàn bộ bằng Docker (MySQL và ứng dụng) để chạy được bằng một cú nhấp; xây dựng bộ kiểm thử tự động chạy trên MySQL thật để bảo đảm các kịch bản luôn tái hiện đúng.

1.3. Phạm vi đề tài

Đề tài giới hạn ở hệ quản trị **MySQL 9.6.0** với storage engine **InnoDB**, chạy trên một máy chủ đơn (không xét nhân bản, phân cụm hay giao tác phân tán). Nghiệp vụ minh họa là một hệ thống đặt vé máy bay đã được giản lược còn ba bảng (chuyến bay, vé, ghế), đủ để tạo ra mọi loại tranh chấp cần nghiên cứu. Về mức cô lập, đề tài khảo sát đủ bốn mức của chuẩn SQL: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ và SERIALIZABLE.

Ứng dụng minh họa phục vụ mục đích học tập và thực nghiệm, chạy cục bộ trên máy cá nhân thông qua Docker; đề tài không hướng tới xây dựng một hệ thống đặt vé hoàn chỉnh (thanh toán, tài khoản người dùng, phân quyền...). Thời gian thực hiện là một học kỳ của học phần Hệ quản trị cơ sở dữ liệu.

1.4. Đối tượng nghiên cứu

Đối tượng nghiên cứu chính của đề tài là **cơ chế điều khiển truy xuất đồng thời của InnoDB**, bao gồm: mô hình giao tác và các mức cô lập; hệ thống khóa (khóa dòng, khóa khoảng trống, next-key lock, khóa ý định, khóa ý định chèn); cơ chế đa phiên bản dựa trên undo log; cơ chế phát hiện và xử lý khóa chết. Bên cạnh đó, đề tài nghiên cứu các **kỹ thuật phía ứng dụng** để xử lý tranh chấp: khóa bi quan với SELECT ... FOR UPDATE, khóa lạc quan với cột phiên bản, câu lệnh UPDATE nguyên tử có điều kiện, và quy ước thứ tự khóa để phòng tránh khóa chết.

1.5. Phương pháp nghiên cứu

Phương pháp thu thập thông tin: đọc và tổng hợp giáo trình cơ sở dữ liệu [1], [2], các công trình kinh điển về xử lý giao tác [3], [6], [7], tài liệu chính thức của MySQL về mô hình khóa và giao tác của InnoDB [4], [8]–[11].

Phương pháp xử lý thông tin: phân tích định tính các lịch giao tác (viết bằng ký hiệu r, w, c, a) để giải thích vì sao một xung đột xảy ra hay không; phân tích định lượng các số đo thu được khi chạy thực nghiệm như thời gian xử lý, số vé bán vượt, số lần chờ khóa, số lần thử lại.

Phương pháp thực nghiệm: xây dựng ứng dụng điều phối nhiều phiên làm việc trên MySQL thật, chạy các kịch bản với thứ tự xen kẽ được kiểm soát, ghi nhận kết quả từng câu lệnh và chụp bảng khóa từ performance_schema sau mỗi bước; so sánh kết quả đo được với lý thuyết. Toàn bộ kịch bản được kiểm chứng lặp lại bằng bộ kiểm thử tự động gồm 151 trường hợp.

1.6. Bố cục đề tài

Phần còn lại của báo cáo tiểu luận được tổ chức như sau.

Chương 2 trình bày cơ sở lý thuyết của đề tài. Chương này bắt đầu từ khái niệm giao tác, các tính chất ACID và lịch giao tác, sau đó phân tích năm hiện tượng bất thường có thể xảy ra khi nhiều giao tác truy xuất đồng thời. Tiếp theo, chương trình bày các kỹ thuật điều khiển đồng thời gồm kỹ thuật khóa, giao thức khóa hai pha, điều khiển đa phiên bản và các cách xử lý khóa chết, cùng với bốn mức cô lập của chuẩn SQL. Phần cuối chương đi sâu

vào MySQL và InnoDB: kiến trúc, các loại khóa, cơ chế đa phiên bản, phát hiện khóa chết và cách giám sát khóa qua `performance_schema`, rồi giới thiệu các công nghệ dùng để xây dựng ứng dụng minh họa.

Chương 3 trình bày đóng góp chính của đề tài là ứng dụng minh họa điều khiển truy xuất đồng thời. Phần phân tích hệ thống xác định yêu cầu, kiến trúc, lược đồ cơ sở dữ liệu, thiết kế năm cặp kịch bản xung đột và bộ điều phối cho phép hai phiên chạy xen kẽ ngay cả khi một phiên bị chặn vì chờ khóa. Phần xây dựng sản phẩm mô tả từng chức năng qua giao diện thực tế, đồng thời phân tích kết quả mà MySQL trả về trong từng kịch bản. Chương kết thúc bằng phần kiểm thử, đánh giá và tổng hợp những điểm MySQL/InnoDB khác với lý thuyết.

Phần Kết luận tổng kết những gì đề tài đã làm được so với mục tiêu, các bài học rút ra và hướng phát triển tiếp theo. Phụ lục cung cấp hướng dẫn cài đặt, lược đồ cơ sở dữ liệu và cấu trúc mã nguồn.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

Để giải thích được vì sao một kịch bản gây ra xung đột và vì sao một thay đổi nhỏ lại ngăn được xung đột đó, đề tài cần dựa trên lý thuyết xử lý giao tác. Chương này trình bày các khái niệm nền tảng theo trình tự: giao tác và lịch giao tác (để mô tả chính xác điều gì xảy ra), các hiện tượng bất thường (để biết cần ngăn chặn điều gì), các kỹ thuật điều khiển đồng thời và mức cô lập (để biết hệ quản trị ngăn chặn bằng cách nào), và cuối cùng là cách MySQL/InnoDB cài đặt các kỹ thuật đó – vì đây chính là hệ quản trị mà ứng dụng minh họa sử dụng.

2.1. Giao tác và lịch giao tác

2.1.1. Khái niệm giao tác

Giao tác (transaction) là một đơn vị xử lý logic gồm một dãy thao tác đọc/ghi trên cơ sở dữ liệu, được hệ quản trị xem như một khối không thể chia cắt: hoặc tất cả các thao tác đều có hiệu lực, hoặc không thao tác nào có hiệu lực [1]. Một giao tác bắt đầu bằng lệnh `START TRANSACTION` (hoặc `BEGIN`) và kết thúc bằng `COMMIT` (ghi nhận vĩnh viễn) hoặc `ROLLBACK` (hủy bỏ toàn bộ). Ví dụ, nghiệp vụ bán một vé máy bay gồm hai thao tác phải đi cùng nhau: giảm số ghế trống của chuyến bay và thêm một dòng vé mới.

```
START TRANSACTION;
UPDATE chuyen_bay SET so_ghe_trong = so_ghe_trong - 1
WHERE ma_chuyen = 1 AND so_ghe_trong > 0;
INSERT INTO ve (ma_chuyen, hanh_khach) VALUES (1, 'Nguyễn An');
COMMIT;
```

Nếu hệ thống gặp sự cố giữa hai câu lệnh, giao tác bị hủy và số ghế trống được khôi phục, không có trạng thái “đã trừ ghế nhưng chưa có vé”.

2.1.2. Các tính chất ACID

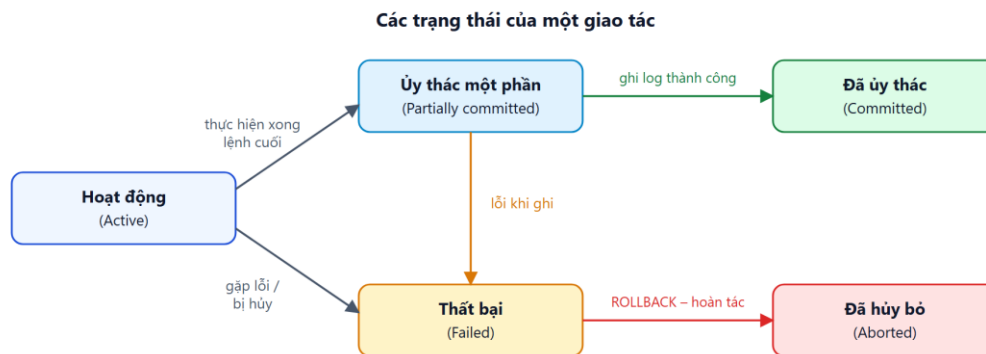
Một hệ quản trị cơ sở dữ liệu hỗ trợ giao tác phải bảo đảm bốn tính chất, thường gọi tắt là ACID (Bảng 2.1). Đề tài tập trung vào tính **cô lập** – tính chất do bộ điều khiển truy xuất đồng thời chịu trách nhiệm.

Bảng 2.1. Bốn tính chất ACID của giao tác

Tính chất	Ý nghĩa	Cơ chế tương ứng trong InnoDB
Atomicity – tính nguyên tử	Tất cả thao tác của giao tác cùng có hiệu lực hoặc cùng bị hủy.	Undo log: hoàn tác các thay đổi khi ROLLBACK hoặc khi giao tác bị hủy.
Consistency – tính nhất quán	Giao tác đưa CSDL từ một trạng thái hợp lệ sang một trạng thái hợp lệ.	Ràng buộc khóa chính, khóa ngoại, CHECK; logic nghiệp vụ trong giao tác.
Isolation – tính cô lập	Các giao tác chạy đồng thời không thấy trạng thái dở dang của nhau.	Hệ thống khóa và MVCC, điều chỉnh bằng mức cô lập.
Durability – tính bền vững	Thay đổi đã COMMIT không bị mất kể cả khi hệ thống gặp sự cố.	Redo log ghi trước (WAL), doublewrite buffer.

2.1.3. Các trạng thái của giao tác

Trong suốt vòng đời, một giao tác đi qua các trạng thái được mô tả ở Hình 2.1. Giao tác ở trạng thái **hoạt động** khi đang thực hiện các thao tác; sau thao tác cuối cùng nó chuyển sang **ủy thác một phần**; khi thông tin cần thiết đã được ghi an toàn vào nhật ký, giao tác **đã ủy thác**. Nếu có lỗi hoặc bị hệ quản trị hủy (chẳng hạn để phá khóa chết), giao tác chuyển sang **thất bại** rồi **đã hủy bỏ** sau khi mọi thay đổi được hoàn tác.



Hình 2.1. Sơ đồ trạng thái của một giao tác

2.1.4. Lịch giao tác và tính khả tuần tự

Khi nhiều giao tác chạy đồng thời, các thao tác của chúng được hệ quản trị thực hiện theo một thứ tự đan xen nào đó gọi là **lịch giao tác** (schedule). Để viết lịch ngắn gọn, giáo trình dùng ký hiệu: $r_1(X)$ – giao tác T_1 đọc đơn vị dữ liệu X ; $w_1(X)$ – T_1 ghi X ; c_1 – T_1

commit; $a_1 - T_1$ hủy bỏ. Ứng dụng minh họa dùng thêm ký hiệu $x_{1_1}(X)$ để chỉ việc T_1 xin khóa ghi trên X .

Một lịch là **tuần tự** nếu các giao tác chạy lần lượt, không đan xen; lịch tuần tự luôn cho kết quả đúng nhưng không tận dụng được tính đồng thời. Một lịch đan xen được gọi là **khả tuần tự** (serializable) nếu kết quả của nó tương đương với một lịch tuần tự nào đó. Hai thao tác của hai giao tác khác nhau **xung đột** nếu cùng truy cập một đơn vị dữ liệu và ít nhất một thao tác là ghi; lịch là **khả tuần tự xung đột** nếu có thể hoán đổi các thao tác không xung đột để biến nó thành một lịch tuần tự [1], [7]. Mục tiêu của bộ điều khiển đồng thời là chỉ cho phép các lịch khả tuần tự (hoặc các lịch đủ an toàn theo mức cô lập đã chọn).

Bảng 2.2 là một lịch đan xen không khả tuần tự kinh điển: cả hai giao tác bán vé đều đọc số ghế trống $X = 7$ rồi cùng ghi 6. Kết quả 6 không tương đương với bất kỳ lịch tuần tự nào (T_1 rồi T_2 hay T_2 rồi T_1 đều cho 5).

Bảng 2.2. Lịch giao tác gây mất cập nhật: $r_1(X)$ $r_2(X)$ $w_1(X)$ c_1 $w_2(X)$ c_2

Thời điểm	T_1 (quầy vé A)	T_2 (ứng dụng di động B)	Giá trị X trong CSDL
t1	$r_1(X) \rightarrow$ đọc được 7		7
t2		$r_2(X) \rightarrow$ đọc được 7	7
t3	$w_1(X) \leftarrow$ ghi $7 - 1 = 6$		6 (chưa commit)
t4	c_1		6
t5		$w_2(X) \leftarrow$ ghi $7 - 1 = 6$	6 (chưa commit)
t6		c_2	6 – lẽ ra phải là 5

2.2. Các vấn đề khi truy xuất đồng thời

Nếu không có cơ chế điều khiển, các lịch đan xen có thể gây ra những hiện tượng bất thường sau. Mỗi hiện tượng được minh họa bằng nghiệp vụ đặt vé, với X là số ghế trống của chuyến VN-808.

2.2.1. Mất cập nhật (Lost update)

Hai giao tác cùng đọc một giá trị, cùng tự tính giá trị mới dựa trên giá trị đã đọc, rồi lần lượt ghi đè. Thay đổi của giao tác ghi trước bị giao tác ghi sau xóa mất (Bảng 2.2). Hậu quả trong nghiệp vụ: bán hai vé nhưng số ghế trống chỉ giảm một, dẫn tới **bán vượt** số ghế. Chuẩn SQL-92 không định nghĩa hiện tượng này một cách hình thức; Berenson và cộng sự

gọi nó là P4 và chỉ ra rằng mức REPEATABLE READ cài đặt bằng khóa sẽ ngăn được nó [3].

2.2.2. Đọc dữ liệu rác (Dirty read)

Giao tác T_1 đọc giá trị mà T_2 đã ghi nhưng **chưa commit**; sau đó T_2 hủy bỏ. T_1 đã dựa vào một dữ liệu chưa từng tồn tại chính thức. Lịch: $w_2(X) \ r_1(X) \ a_2$. Ví dụ: hệ thống đang giữ chỗ tạm thời đặt số ghế trống về 0, giao tác khác đọc thấy “hết vé” và từ chối khách, nhưng sau đó việc giữ chỗ bị hủy.

2.2.3. Đọc không lặp lại (Non-repeatable read)

Trong cùng một giao tác, T_1 đọc một dòng hai lần nhưng nhận hai giá trị khác nhau vì giữa hai lần đọc, T_2 đã sửa và commit dòng đó. Lịch: $r_1(X) \ w_2(X) \ c_2 \ r_1(X)$. Hiện tượng này làm các phép tính dựa trên nhiều lần đọc (chẳng hạn lập báo cáo) trở nên mâu thuẫn.

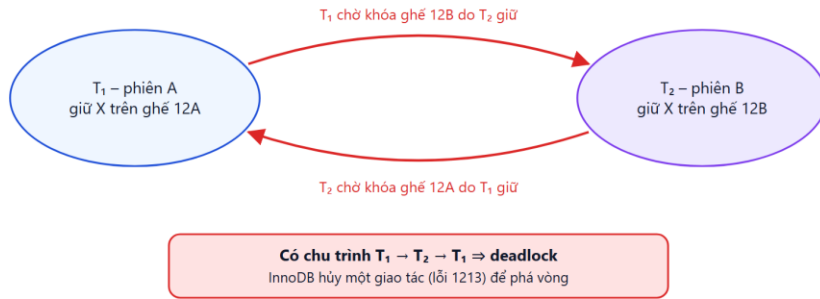
2.2.4. Bóng ma (Phantom read)

T_1 đọc một **tập các dòng** thỏa một điều kiện (một vị từ, ký hiệu P), chẳng hạn “mọi vé của chuyến VN-808”. Trong lúc T_1 chưa kết thúc, T_2 **chèn thêm** một dòng thỏa điều kiện đó và commit. Lần đọc sau, T_1 thấy xuất hiện thêm một dòng “bóng ma”. Khác với đọc không lặp lại (dòng cũ bị sửa), bóng ma liên quan tới dòng **mới**, nên không thể ngăn chỉ bằng cách khóa các dòng đang tồn tại.

2.2.5. Khóa chết (Deadlock)

Khóa chết xảy ra khi các giao tác chờ nhau thành một vòng tròn: T_1 giữ tài nguyên X và chờ Y, trong khi T_2 giữ Y và chờ X. Không giao tác nào tự tiến lên được (Hình 2.2). Khóa chết không làm sai dữ liệu nhưng làm hệ thống treo nếu không được phát hiện; hệ quản trị phải chọn hủy một giao tác (gọi là nạn nhân) để phá vòng chờ, và ứng dụng phía nạn nhân phải chạy lại giao tác.

Đồ thị chờ (wait-for graph) của kịch bản deadlock



Hình 2.2. Đồ thị chờ của hai giao tác bị khóa chết

2.3. Các kỹ thuật điều khiển truy xuất đồng thời

2.3.1. Kỹ thuật khóa

Kỹ thuật khóa (locking) gắn với mỗi đơn vị dữ liệu một khóa mà giao tác phải xin trước khi truy cập. Có hai loại khóa cơ bản: **khóa chia sẻ (S)** cho phép đọc – nhiều giao tác có thể cùng giữ; **khóa độc quyền (X)** cho phép ghi – chỉ một giao tác được giữ và loại trừ mọi khóa khác. Để kết hợp khóa ở nhiều mức (bảng và dòng) mà không phải duyệt từng dòng, các hệ quản trị dùng thêm **khóa ý định IS** và **IX** đặt ở mức bảng trước khi khóa dòng bên trong [4]. Ma trận tương thích của các khóa mức bảng trong InnoDB được cho ở Bảng 2.3: một yêu cầu khóa chỉ được cấp nếu tương thích với mọi khóa mà giao tác khác đang giữ, nếu không giao tác phải chờ.

Bảng 2.3. Ma trận tương thích khóa mức bảng của InnoDB

Yêu cầu \ Đang giữ	X	IX	S	IS
X	Xung đột	Xung đột	Xung đột	Xung đột
IX	Xung đột	Tương thích	Xung đột	Tương thích
S	Xung đột	Xung đột	Tương thích	Tương thích
IS	Xung đột	Tương thích	Tương thích	Tương thích

2.3.2. Giao thức khóa hai pha (2PL)

Chỉ khóa thôi chưa đủ để bảo đảm khả tuần tự – còn phụ thuộc vào thời điểm nhả khóa. **Giao thức khóa hai pha** chia giao tác thành pha tăng trưởng (chỉ xin khóa) và pha thu hẹp (chỉ nhả khóa); một khi đã nhả một khóa thì không được xin thêm khóa nào. Mọi lịch tuân theo 2PL đều khả tuần tự xung đột [7]. Biến thể **2PL nghiêm ngặt** (strict 2PL)

giữ mọi khóa ghi cho tới khi commit hoặc rollback, nhờ đó tránh được hủy dây chuyền; InnoDB áp dụng cách này – khóa dòng chỉ được nhả khi giao tác kết thúc. Nhược điểm của 2PL là giảm tính đồng thời và có thể gây khóa chết.

2.3.3. Điều khiển đồng thời đa phiên bản (MVCC)

MVCC lưu nhiều phiên bản của mỗi dòng dữ liệu. Khi một giao tác ghi, phiên bản cũ được giữ lại; giao tác khác cần đọc sẽ được cung cấp phiên bản phù hợp với thời điểm của nó (một **ảnh chụp – snapshot**) thay vì phải chờ khóa. Nhờ vậy, **người đọc không chặn người ghi và người ghi không chặn người đọc**. MVCC thường được kết hợp với khóa: đọc thường dùng snapshot, còn ghi và đọc có khóa vẫn dùng khóa để tuần tự hóa. Cách InnoDB cài đặt MVCC được trình bày ở mục 2.5.4.

2.3.4. Khóa bi quan và khóa lạc quan

Ở mức ứng dụng, có hai triết lý xử lý tranh chấp khi đọc dữ liệu rồi ghi dựa trên giá trị đã đọc (Bảng 2.4). Ngoài ra, nếu toàn bộ điều kiện có thể viết trong **một câu UPDATE có điều kiện** (ví dụ SET so_ghe_trong = so_ghe_trong - 1 WHERE ... AND so_ghe_trong > 0) thì việc đọc và ghi diễn ra nguyên tử bên trong hệ quản trị, không còn khe hở giữa đọc và ghi.

Bảng 2.4. So sánh khóa bi quan và khóa lạc quan

Tiêu chí	Khóa bi quan (pessimistic)	Khóa lạc quan (optimistic)
Giả định	Xung đột thường xảy ra, phải ngăn từ đầu.	Xung đột hiếm, chỉ cần phát hiện khi ghi.
Cách làm	Khóa dòng ngay khi đọc: SELECT ... FOR UPDATE.	Đọc kèm cột version; khi ghi kiểm tra WHERE version = giá trị đã đọc và tăng version.
Khi có tranh chấp	Giao tác sau phải chờ giao tác trước commit.	Câu UPDATE không tác động dòng nào; ứng dụng phải đọc lại và thử lại.
Ưu điểm	Đơn giản, không phải thử lại.	Không giữ khóa trong lúc người dùng cân nhắc.
Nhược điểm	Giữ khóa lâu làm giảm tính đồng thời, có thể khóa chết.	Tranh chấp cao thì thử lại rất nhiều, có thể bỏ cuộc.

2.3.5. Xử lý khóa chết

Có ba hướng xử lý khóa chết: **phòng ngừa** – buộc mọi giao tác xin khóa theo cùng một thứ tự toàn cục (ví dụ luôn khóa ghế có mã nhỏ trước) thì không thể hình thành vòng chờ; **phát hiện và phá vỡ** – hệ quản trị duy trì đồ thị chờ, khi phát hiện chu trình thì chọn một giao tác làm nạn nhân và hủy nó; và **hết thời gian chờ** – giao tác chờ khóa quá một ngưỡng thời gian thì bị báo lỗi. InnoDB dùng đồng thời phát hiện vòng chờ và hết thời gian chờ (mục 2.5.5).

2.4. Mức cô lập giao tác theo chuẩn SQL

Bảo đảm khả tuần tự tuyệt đối có chi phí cao, nên chuẩn SQL định nghĩa bốn **mức cô lập** cho phép đánh đổi giữa tính đúng đắn và hiệu năng. Mỗi mức được định nghĩa bằng những hiện tượng mà nó cho phép xảy ra (Bảng 2.5) [5]. Người lập trình chọn mức cô lập cho từng phiên bằng lệnh SET SESSION TRANSACTION ISOLATION LEVEL

Berenson và cộng sự [3] chỉ ra rằng định nghĩa dựa trên hiện tượng của chuẩn SQL còn mơ hồ và bỏ sót một số bất thường như mất cập nhật (P4) hay lệch ghi (write skew); đồng thời mỗi hệ quản trị cài đặt các mức cô lập theo cách riêng (bằng khóa hay bằng snapshot), nên hành vi thực tế có thể khác bảng lý thuyết. Đây chính là điều ứng dụng minh họa đo đạc và kiểm chứng ở Chương 3.

Bảng 2.5. Các mức cô lập và hiện tượng được phép xảy ra theo chuẩn SQL

Mức cô lập	Đọc dữ liệu rác	Đọc không lặp lại	Bóng ma
READ UNCOMMITTED	Có thể	Có thể	Có thể
READ COMMITTED	Không	Có thể	Có thể
REPEATABLE READ	Không	Không	Có thể
SERIALIZABLE	Không	Không	Không

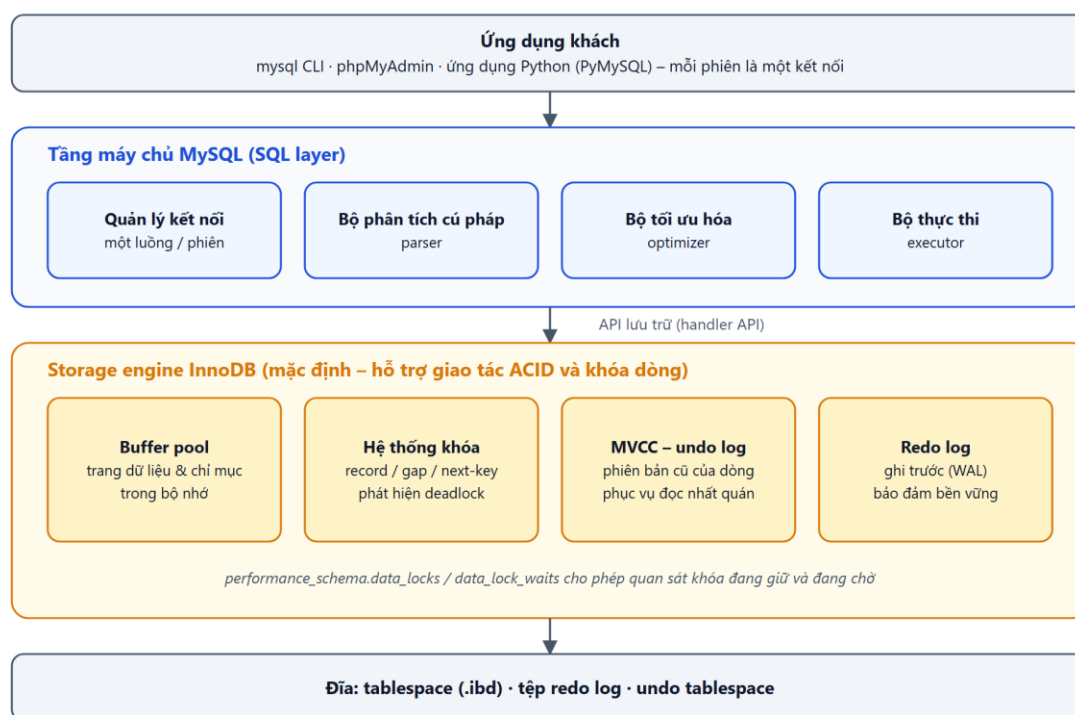
2.5. Hệ quản trị CSDL MySQL và storage engine InnoDB

2.5.1. Lịch sử hình thành và kiến trúc

MySQL được công ty MySQL AB (Thụy Điển) phát hành lần đầu năm 1995 và nhanh chóng trở thành hệ quản trị cơ sở dữ liệu mã nguồn mở phổ biến cho các ứng dụng web. Năm 2008, MySQL AB được Sun Microsystems mua lại; năm 2010, Oracle mua Sun và

tiếp quản MySQL. Storage engine **InnoDB** do Innobase Oy phát triển, trở thành engine mặc định từ phiên bản 5.5 (2010) thay cho MyISAM vốn không hỗ trợ giao tác. Từ năm 2023, Oracle phát hành MySQL theo hai dòng: bản hỗ trợ dài hạn (LTS) và bản Innovation; ứng dụng minh họa dùng MySQL 9.6.0.

MySQL có kiến trúc phân tầng (Hình 2.3): tầng máy chủ lo kết nối, phân tích cú pháp, tối ưu và thực thi câu lệnh; tầng lưu trữ gồm các storage engine giao tiếp qua một API chung. Chỉ engine nào hỗ trợ giao tác mới có điều khiển truy xuất đồng thời – vì vậy mọi bảng trong đề tài đều khai báo `ENGINE=InnoDB`. Điểm mạnh của MySQL/InnoDB là phổ biến, miễn phí, hỗ trợ đầy đủ ACID, khóa mức dòng và MVCC; điểm yếu là một số hành vi mức cô lập khác chuẩn SQL (như sẽ thấy ở Chương 3), và mức mặc định REPEATABLE READ không tự phát hiện xung đột ghi như một số hệ quản trị khác.



Hình 2.3. Kiến trúc phân tầng của MySQL và các thành phần chính của InnoDB

2.5.2. Mô hình giao tác của InnoDB

Mỗi kết nối tới MySQL là một **phiên** (session); hai giao tác đồng thời luôn thuộc hai phiên khác nhau. Mặc định MySQL bật chế độ autocommit (mỗi câu lệnh là một giao tác),

và mức cô lập mặc định của InnoDB là **REPEATABLE READ** (máy chủ trong đề tài báo REPEATABLE-READ).

InnoDB phân biệt hai cách đọc [8]. **Đọc nhất quán** (consistent read) là câu SELECT thường: nó đọc trên snapshot, không đặt khóa và không bao giờ bị chặn. Ở REPEATABLE READ, snapshot được tạo ở lần đọc đầu tiên và dùng lại suốt giao tác; ở READ COMMITTED, mỗi câu SELECT lấy một snapshot mới. **Đọc có khóa** (locking read) gồm SELECT ... FOR UPDATE, SELECT ... FOR SHARE và các câu UPDATE, DELETE: chúng luôn đọc **bản mới nhất** (đọc hiện hành – current read) và đặt khóa lên các bản ghi đã duyệt. Riêng ở mức SERIALIZABLE, InnoDB tự chuyển mọi SELECT thường thành SELECT ... FOR SHARE khi autocommit tắt.

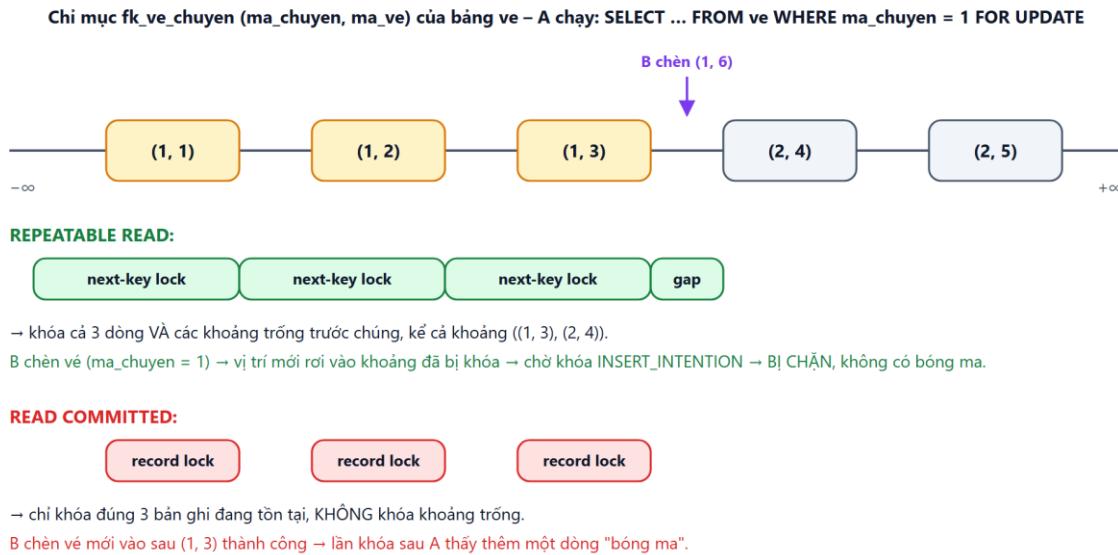
Hệ quả quan trọng: câu UPDATE ở REPEATABLE READ ghi lên bản mới nhất mà không kiểm tra dòng đó đã bị giao tác khác sửa sau thời điểm snapshot hay chưa. Đây là nguyên nhân MySQL vẫn dễ xảy ra mất cập nhật ở mức này.

2.5.3. Các loại khóa trong InnoDB

Khóa dòng của InnoDB thực chất là khóa trên **bản ghi chỉ mục**. Các loại khóa chính gồm [4]:

- **Record lock** – khóa đúng một bản ghi chỉ mục (trong performance_schema hiện là X, REC_NOT_GAP hoặc S, REC_NOT_GAP).
- **Gap lock** – khóa **khoảng trống** giữa hai bản ghi chỉ mục, không khóa bản ghi nào (X, GAP). Gap lock chỉ có tác dụng ngăn chèn dữ liệu mới vào khoảng đó; các gap lock không xung đột với nhau.
- **Next-key lock** – record lock trên một bản ghi cộng với gap lock trên khoảng trống ngay trước nó (X). Ở REPEATABLE READ, đọc có khóa theo điều kiện phạm vi dùng next-key lock để chặn bóng ma.
- **Insert intention lock** – một loại gap lock do câu INSERT đặt trước khi chèn (X, GAP, INSERT_INTENTION); nó phải chờ nếu khoảng trống đang bị giao tác khác giữ gap lock.
- **Khóa ý định IS/IX** – đặt ở mức bảng trước khi khóa dòng bên trong.

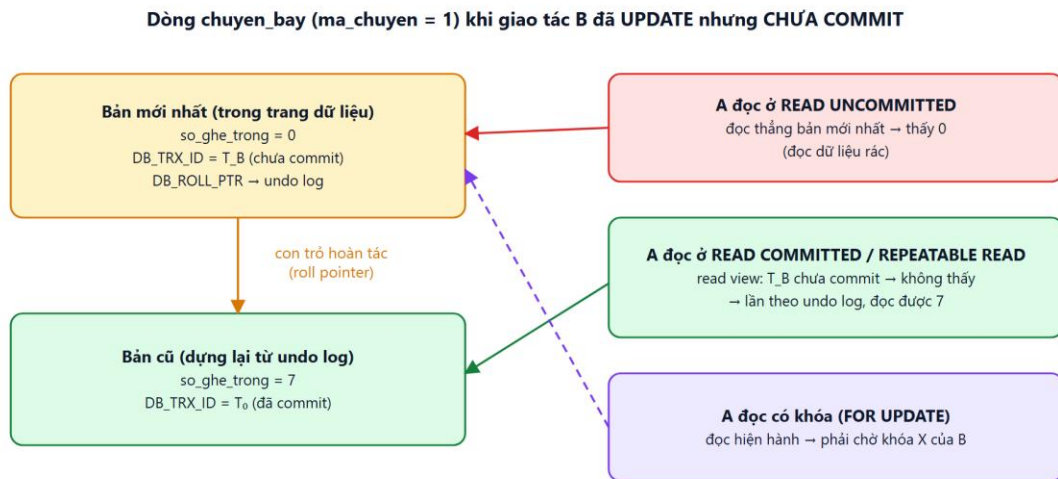
Hình 2.4 minh họa sự khác biệt giữa REPEATABLE READ và READ COMMITTED khi giao tác A chạy `SELECT ... FROM ve WHERE ma_chuyen = 1 FOR UPDATE`. Ở READ COMMITTED, InnoDB tắt gap lock cho các phép tìm kiếm thông thường nên chỉ khóa các bản ghi đang tồn tại – đó là lý do bóng ma xuất hiện ở mức này.



Hình 2.4. Record lock, gap lock và next-key lock trên chỉ mục `ma_chuyen`

2.5.4. Cơ chế đa phiên bản (MVCC) trong InnoDB

Mỗi dòng trong InnoDB có các trường ẩn, trong đó `DB_TRX_ID` lưu mã giao tác cuối cùng đã sửa dòng và `DB_ROLL_PTR` trỏ tới bản ghi trong **undo log** chứa phiên bản trước [9]. Khi một giao tác cần đọc nhất quán, InnoDB tạo một **read view** – danh sách các giao tác đang hoạt động tại thời điểm đó. Nếu phiên bản hiện tại do một giao tác chưa commit (hoặc commit sau read view) tạo ra, InnoDB lần theo con trỏ hoàn tác để dựng lại phiên bản cũ phù hợp (Hình 2.5). Ở READ UNCOMMITTED, InnoDB bỏ qua bước này và đọc thẳng bản mới nhất – đó chính là đọc dữ liệu rác.



Nhờ nhiều phiên bản, người đọc không phải chờ người ghi (trừ khi đọc có khóa).

Hình 2.5. MVCC: cùng một dòng, các cách đọc khác nhau thấy các phiên bản khác nhau

2.5.5. Phát hiện và xử lý khóa chết trong InnoDB

Khi tham số `innodb_deadlock_detect` bật (mặc định), InnoDB kiểm tra đồ thị chờ mỗi khi một giao tác phải chờ khóa. Nếu phát hiện chu trình, InnoDB **ngay lập tức** hủy một giao tác – ưu tiên giao tác “nhỏ” hơn, tính theo số dòng đã chèn, sửa, xóa – và trả về lỗi **1213** (ER_LOCK_DEADLOCK) [10]. Ngoài ra, giao tác chờ khóa lâu hơn `innodb_lock_wait_timeout` sẽ nhận lỗi **1205**. Trong đề tài, máy chủ đặt `innodb_lock_wait_timeout = 10` giây và bật `innodb_print_all_deadlocks` để ghi mọi khóa chết vào nhật ký.

2.5.6. Giám sát khóa qua `performance_schema`

Từ MySQL 8.0, thông tin khóa của InnoDB được cung cấp qua hai bảng `performance_schema.data_locks` (mọi khóa đang giữ hoặc đang chờ) và `performance_schema.data_lock_waits` (cặp yêu cầu đang chờ – khóa đang chặn) [11]. Các cột quan trọng gồm `OBJECT_NAME` (bảng), `INDEX_NAME` (chỉ mục), `LOCK_TYPE` (TABLE / RECORD), `LOCK_MODE` (chế độ khóa, ví dụ X,GAP), `LOCK_STATUS` (GRANTED / WAITING) và `LOCK_DATA` (giá trị khóa của bản ghi). Ứng dụng minh họa đọc hai bảng này để hiển thị khóa tại từng bước:

```

SELECT t.PROCESSLIST_ID AS ma_conn, l.ENGINE_TRANSACTION_ID AS trx,
       l.OBJECT_NAME AS bang, l.INDEX_NAME AS chi_muc, l.LOCK_TYPE AS loai_khoa,
       l.LOCK_MODE AS che_do_khoa, l.LOCK_STATUS AS trang_thai,
       l.LOCK_DATA AS du_lieu
FROM performance_schema.data_locks l
JOIN performance_schema.threads t ON t.THREAD_ID = l.THREAD_ID
WHERE l.OBJECT_SCHEMA = DATABASE();

```

2.6. Công nghệ xây dựng ứng dụng minh họa

Ứng dụng minh họa được xây dựng với các công nghệ trong Bảng 2.6. Tiêu chí lựa chọn là gọn nhẹ, phổ biến, cho phép mở nhiều kết nối MySQL song song và dễ triển khai trên máy của mọi thành viên.

Bảng 2.6. Các công nghệ sử dụng

Thành phần	Công nghệ	Vai trò
Hệ quản trị CSDL	MySQL 9.6.0, InnoDB	Đối tượng nghiên cứu; chạy trong container Docker.
Máy chủ ứng dụng	Python 3.13, FastAPI, Uvicorn	API REST điều phối các kịch bản, trả kết quả dạng JSON.
Kết nối CSDL	PyMySQL	Mỗi phiên/khách là một kết nối riêng chạy trên một luồng (thread) riêng.
Giao diện	HTML, CSS, JavaScript thuần	4 tab chức năng, hoạt hình lịch giao tác, biểu đồ SVG.
Triển khai	Docker, Docker Compose	Đóng gói MySQL và ứng dụng; chạy bằng tệp khai-dong.bat.
Kiểm thử	pytest, httpx	151 trường hợp kiểm thử chạy trên MySQL thật.
Quản lý mã nguồn	Git, GitHub	Chia sẻ mã nguồn cho các thành viên nhóm.

CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG VÀ XÂY DỰNG SẢN PHẨM

3.1. Phân tích hệ thống

Dựa trên lý do chọn đề tài và mục tiêu ở Chương 1, ứng dụng minh họa phải trả lời được ba câu hỏi cho người học: xung đột xảy ra như thế nào (thấy được lịch giao tác và dữ liệu tại từng bước), vì sao một thay đổi nhỏ lại ngăn được xung đột (so sánh kịch bản sai và đúng), và cái giá của từng cách xử lý là gì (đo đạc khi nhiều giao tác cùng tranh chấp). Yêu cầu xuyên suốt là mọi kết quả phải đến từ MySQL thật, không được mô phỏng.

3.1.1. Yêu cầu chức năng

Bảng 3.1. Yêu cầu chức năng

Mã	Chức năng	Mô tả
CN01	Năm xung đột: sai và đúng	Chọn một trong 5 xung đột, chạy kịch bản SAI hoặc ĐÚNG trên MySQL; hiển thị lịch giao tác T_1 T_2 , kết quả từng câu lệnh, dữ liệu và khóa.
CN02	Phát lại dạng hoạt hình	Phát/dừng, đi từng bước, nhảy tới bước bất kỳ, chọn tốc độ; giải thích bằng lời từng bước.
CN03	So sánh hai kịch bản	Đặt cạnh nhau mức cô lập, câu lệnh then chốt, lịch thực thi, số lần bị chặn, thời gian chờ khóa và lỗi của kịch bản sai và đúng.
CN04	Nhiều khách đặt vé	1–60 khách (mỗi khách một kết nối) cùng đặt vé với 4 cách xử lý hoặc kịch bản deadlock; biểu đồ diễn biến, sơ đồ ghế, bảng khóa, nhật ký SQL.
CN05	Phòng thí nghiệm mức cô lập	Chạy 6 thí nghiệm ở 4 mức cô lập (24 lần) và lập ma trận đối chiếu với lý thuyết chuẩn SQL; xem chi tiết từng thí nghiệm.
CN06	So sánh hiệu năng	Chạy 4 cách xử lý nhiều lần, lấy trung bình; khảo sát khi số khách tăng dần.
CN07	Giám sát khóa	Đọc performance_schema.data_locks và data_lock_waits để biết ai giữ, ai chờ khóa gì.

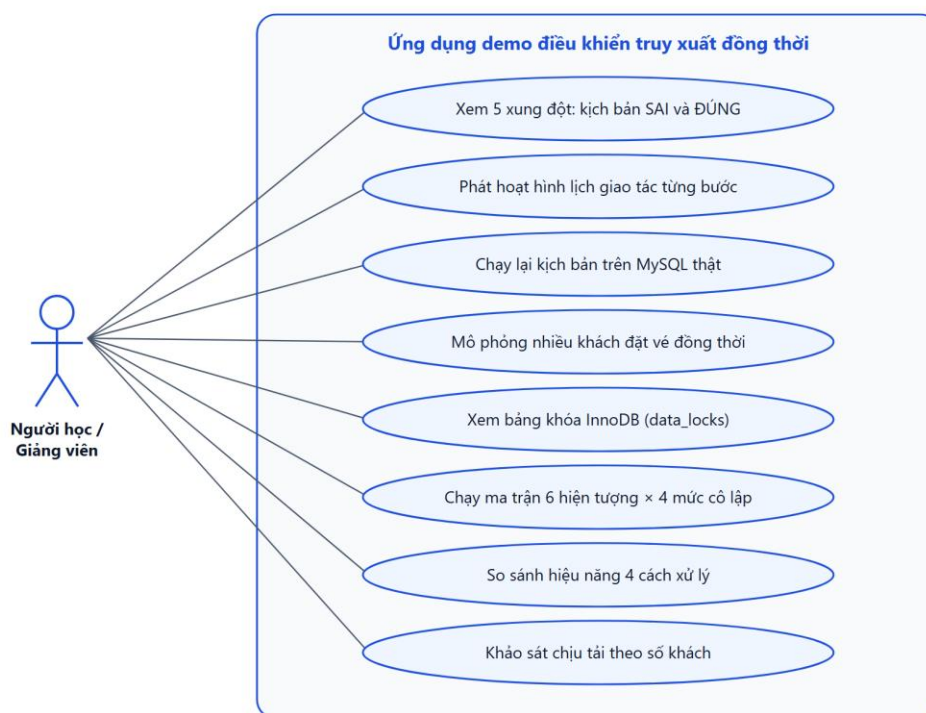
3.1.2. Yêu cầu phi chức năng

Bảng 3.2. Yêu cầu phi chức năng

Yêu cầu	Mô tả
Trung thực	Mọi số liệu, kết quả, bảng khóa lấy từ MySQL thật; giao diện chỉ phát lại những gì đã xảy ra.
Tái hiện được	Kịch bản xung đột cho cùng kết quả ở mọi lần chạy (thứ tự xen kẽ được cố định).
Dễ triển khai	Chỉ cần Docker Desktop; nhấp đúp khai-dong.bat là chạy, không đụng MySQL có sẵn trên máy (cổng 3310).
An toàn dữ liệu demo	Tại một thời điểm chỉ một kịch bản được chạy (trả mã 409 nếu đang bận).
Kiểm thử tự động	Bộ kiểm thử chạy trên MySQL thật, chạy lặp nhiều lần không chập chờn.
Dễ hiểu	Giao diện tiếng Việt, ký hiệu lịch theo giáo trình, chú giải và giải thích từng bước.

3.1.3. Sơ đồ use case

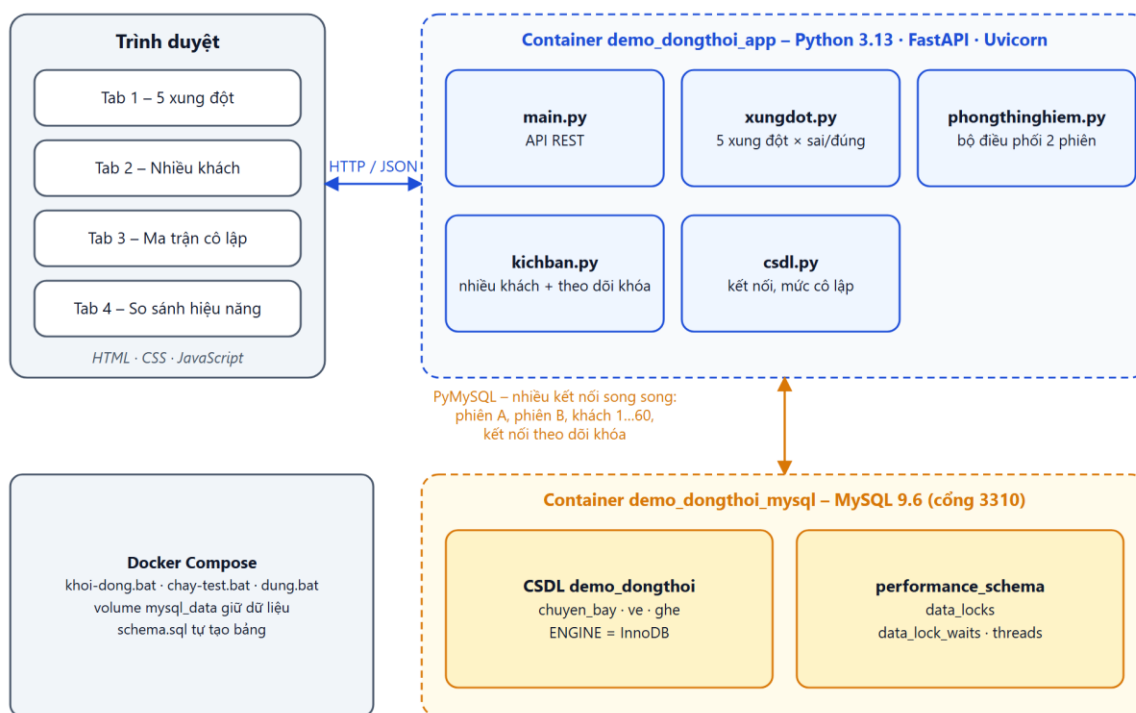
Ứng dụng có một tác nhân là người học hoặc giảng viên. Hình 3.1 liệt kê các chức năng mà tác nhân sử dụng; mỗi nhóm chức năng tương ứng với một tab của giao diện.



Hình 3.1. Sơ đồ use case của ứng dụng minh họa

3.1.4. Kiến trúc hệ thống

Hệ thống gồm ba thành phần (Hình 3.2). **Trình duyệt** hiển thị giao diện bốn tab, gọi API bằng HTTP và nhận kết quả dạng JSON. **Container ứng dụng** chạy FastAPI, gồm các mô-đun: `main.py` (khai báo API), `xungdot.py` (định nghĩa 5 xung đột $\times$ 2 kịch bản), `phongthinghiem.py` (bộ điều phối hai phiên và các thí nghiệm mức cô lập), `kichban.py` (nhiều khách đặt vé, theo dõi khóa, so sánh hiệu năng) và `csdl.py` (quản lý kết nối, mức cô lập). **Container MySQL** chạy MySQL 9.6.0, ánh xạ ra cổng 3310 để không xung đột với MySQL có sẵn trên máy. Mỗi phiên A, B hay mỗi khách hàng là một kết nối PyMySQL riêng, vì trong MySQL một giao tác gắn với đúng một kết nối.



Hình 3.2. Kiến trúc tổng thể của ứng dụng minh họa

Các API chính của ứng dụng được liệt kê trong Bảng 3.3.

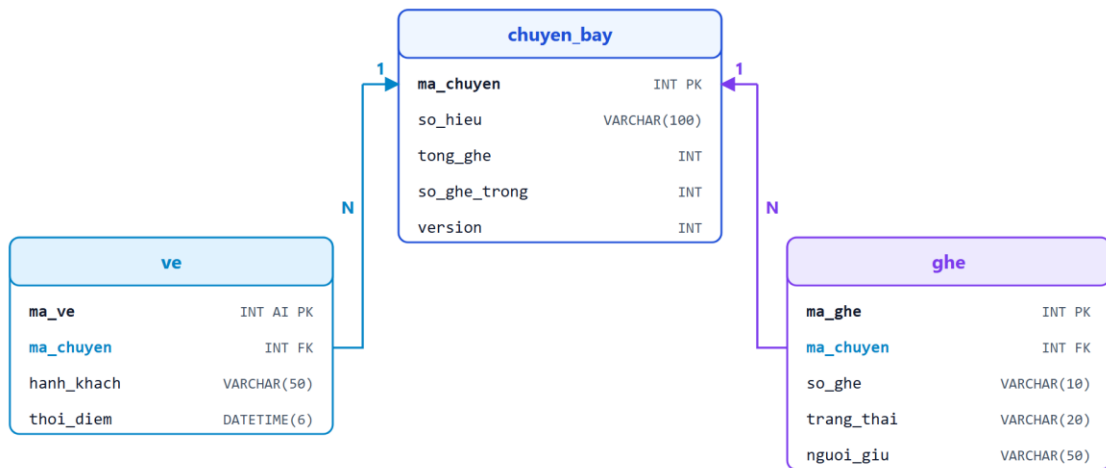
Bảng 3.3. Các API của ứng dụng

Phương thức	Đường dẫn	Chức năng
GET	/api/thong-tin	Thông tin máy chủ MySQL (phiên bản, engine, mức cô lập mặc định...).

Phương thức	Đường dẫn	Chức năng
GET	/api/xung-dot	Danh sách 5 xung đột kèm mã nguồn SQL của từng phiên.
POST	/api/xung-dot/chay	Chạy một kịch bản (mã xung đột, sai/đúng), trả về nhật ký từng bước.
POST	/api/chay	Nhiều khách đặt vé với một cách xử lý.
POST	/api/thi-nghiem/chay	Chạy một thí nghiệm ở một mức cô lập.
POST	/api/thi-nghiem/ma-tran	Chạy toàn bộ 6 thí nghiệm $\times$ 4 mức cô lập.
POST	/api/so-sanh	So sánh 4 cách xử lý, lặp nhiều lần.
POST	/api/khao-sat	Khảo sát khi số khách tăng dần.

3.1.5. Thiết kế cơ sở dữ liệu

Cơ sở dữ liệu demo_dongthoi gồm ba bảng, đều dùng ENGINE=InnoDB (Hình 3.3). Bảng chuyen_bay lưu số ghế trống của từng chuyến – đây là **điểm tranh chấp chính**: mọi giao tác bán vé đều đọc và ghi cột so_ghe_trong của cùng một dòng. Cột version phục vụ khóa lạc quan. Bảng ve lưu mỗi vé đã bán và có chỉ mục fk_ve_chuyen trên cột ma_chuyen – chỉ mục này là nơi xuất hiện gap lock trong kịch bản bóng ma. Bảng ghe lưu hai ghế 12A, 12B dùng cho kịch bản khóa chết.



Mọi bảng dùng ENGINE = InnoDB; so_ghe_trong là điểm tranh chấp chính, version phục vụ khóa lạc quan.

Hình 3.3. Lược đồ cơ sở dữ liệu demo_dongthoi

Bảng 3.4. Mô tả các cột của ba bảng

Bảng	Cột	Kiểu	Ý nghĩa
chuyen_bay	ma_chuyen (PK)	INT	Mã chuyến bay

Bảng	Cột	Kiểu	Ý nghĩa
	so_hieu	VARCHAR(100)	Số hiệu, ví dụ VN-808
	tong_ghe	INT	Tổng số ghế mở bán
	so_ghe_trong	INT	Số ghế còn trống – dữ liệu bị tranh chấp
	version	INT	Phiên bản dòng, tăng mỗi lần ghi (khóa lạc quan)
ve	ma_ve (PK)	INT AUTO_INCREMENT	Mã vé
	ma_chuyen (FK)	INT	Chuyến bay của vé
	hanh_khach	VARCHAR(50)	Tên hành khách
	thoi_diem	DATETIME(6)	Thời điểm bán, chính xác tới micro giây
ghe	ma_ghe (PK)	INT	Mã ghế
	ma_chuyen (FK)	INT	Chuyến bay của ghế
	so_ghe	VARCHAR(10)	Số ghế: 12A, 12B
	trang_thai	VARCHAR(20)	'trong' hoặc 'dang_giu'
	nguoai_giu	VARCHAR(50)	Phiên đang giữ ghế

Trước mỗi kịch bản, ứng dụng xóa và nạp lại dữ liệu ban đầu: chuyến VN-808 (mã 1) có 10 ghế, còn trống 7; chuyến VN-216 (mã 2) còn trống 8; bảng ve có 5 vé (An, Bình, Chi thuộc VN-808; Dũng, Giang thuộc VN-216); bảng ghe có hai ghế 12A, 12B đều trống. Nhờ vậy mọi kịch bản xuất phát từ cùng một trạng thái.

3.1.6. Thiết kế năm cặp kịch bản xung đột

Nguyên tắc thiết kế quan trọng nhất là: **kịch bản ĐÚNG chỉ khác kịch bản SAI đúng một điểm** – một mệnh đề, một mức cô lập hoặc một thứ tự khóa. Hai kịch bản chạy trên cùng dữ liệu, cùng thứ tự xen kẽ các bước, nên người xem biết chắc điểm khác biệt đó chính là nguyên nhân (hoặc cách sửa). Trên giao diện, dòng then chốt được đánh dấu “← nguyên nhân” ở kịch bản sai và “← cách sửa” ở kịch bản đúng. Bảng 3.5 tóm tắt thiết kế.

Bảng 3.5. Thiết kế năm cặp kịch bản sai / đúng

#	Xung đột	Kịch bản SAI	Kịch bản ĐÚNG	Khác biệt duy nhất
1	Mất cập nhật	REPEATABLE READ, SELECT thường rồi UPDATE ghi đè	REPEATABLE READ, SELECT ... FOR UPDATE	Thêm FOR UPDATE

#	Xung đột	Kịch bản SAI	Kịch bản ĐÚNG	Khác biệt duy nhất
2	Đọc dữ liệu rác	READ UNCOMMITTED	READ COMMITTED	Mức cô lập
3	Đọc không lặp lại	READ COMMITTED	REPEATABLE READ	Mức cô lập
4	Bóng ma	READ COMMITTED + FOR UPDATE	REPEATABLE READ + FOR UPDATE	Mức cô lập (gap lock)
5	Khóa chết	A khóa 12A → 12B, B khóa 12B → 12A	Cả hai khóa 12A → 12B	Thứ tự khóa

Mỗi kịch bản là một danh sách bước có thứ tự, mỗi bước gồm mã bước, phiên thực hiện, câu SQL và mô tả. Ví dụ kịch bản SAI của xung đột mất cập nhật được khai báo trong `xungdot.py` như sau (bước K1 là bước kiểm tra, không thuộc giao tác nào):

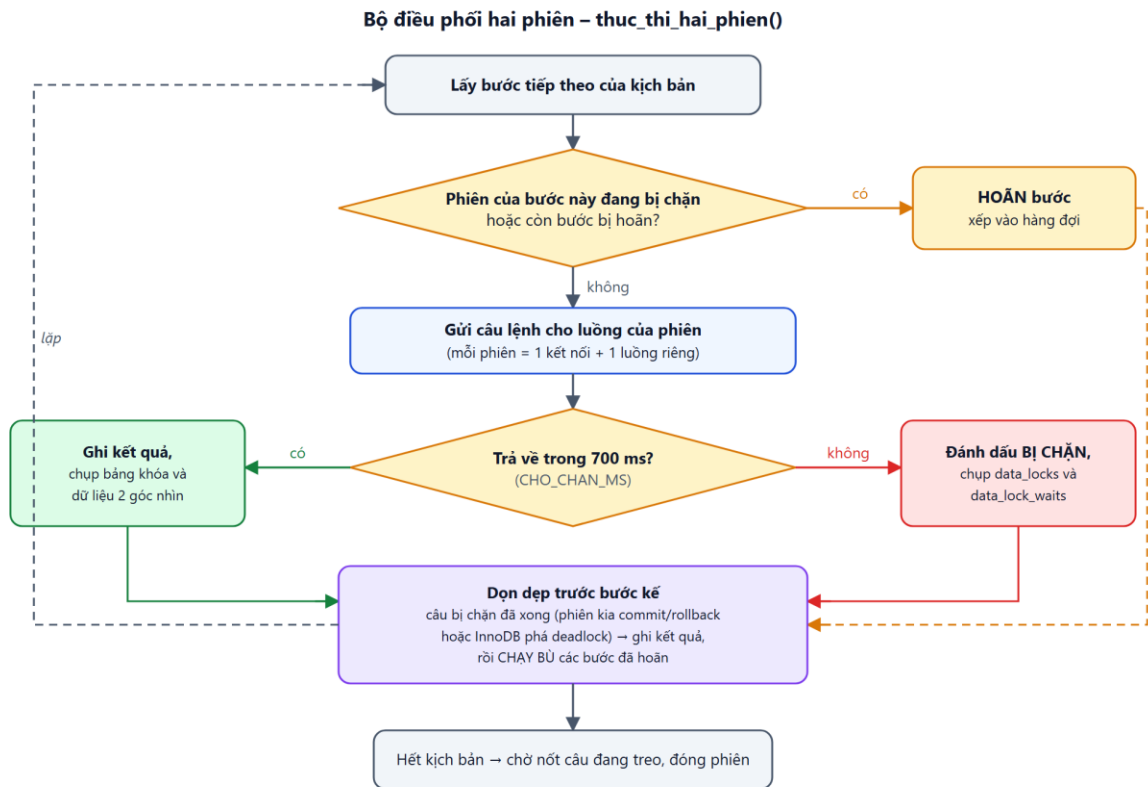
```
Buoc("A1", "A", "START TRANSACTION", "A mở giao tác"),
Buoc("B1", "B", "START TRANSACTION", "B mở giao tác"),
Buoc("A2", "A", SQL_DOC_GHE, "A đọc số ghế còn - đọc thường, không khóa",
    diem_khac=True),
Buoc("B2", "B", SQL_DOC_GHE, "B cũng đọc số ghế còn", diem_khac=True),
Buoc("A3", "A", _ghi_tru_mot("A2"), "A bán 1 vé: ghi lại giá trị đã đọc - 1"),
Buoc("A4", "A", "COMMIT", "A commit"),
Buoc("B3", "B", _ghi_tru_mot("B2"), "B bán 1 vé: ghi lại giá trị đã đọc - 1"),
Buoc("B4", "B", "COMMIT", "B commit"),
Buoc("K1", "A", SQL_DOC_GHE, "Kiểm tra: lẽ ra phải còn 5 ghế"),
```

3.1.7. Bộ điều phối hai phiên

Thách thức kỹ thuật lớn nhất của ứng dụng là chạy hai phiên xen kẽ đúng thứ tự trong khi một câu lệnh có thể bị InnoDB **chặn** vì chờ khóa: phiên bị chặn treo lại, nhưng phiên kia vẫn phải chạy tiếp được (nếu không, khóa sẽ không bao giờ được nhả). Bộ điều phối `thuc_thi_hai_phien()` giải quyết như Hình 3.4: mỗi phiên có một kết nối và một luồng thực thi riêng; bộ điều phối gửi câu lệnh rồi chờ tối đa 700 ms. Nếu câu lệnh chưa trả về, bước đó được đánh dấu **bị chặn** và bảng khóa được chụp ngay lúc ấy; các bước sau của cùng phiên được **hoãn** lại vì một kết nối không thể gửi câu lệnh mới khi câu trước chưa xong. Khi câu bị chặn hoàn tất (do phiên kia commit/rollback hoặc do InnoDB phá khóa chết), kết quả được ghi nhận và các bước đã hoãn được **chạy bù**.

Sau mỗi sự kiện, bộ điều phối chụp dữ liệu theo **hai góc nhìn** bằng hai kết nối riêng không đặt khóa: một kết nối ở READ COMMITTED (thấy bản đã commit) và một kết nối ở READ UNCOMMITTED (thấy bản mới nhất, kể cả bản nháp chưa commit). Giao diện

hiển thị hai giá trị này cạnh nhau: giá trị gạch ngang là bản đã commit, giá trị tô vàng là bản nháp. Nhờ đó người xem thấy rõ dữ liệu “dở dang” của từng giao tác.



Hình 3.4. Lưu đồ bộ điều phối hai phiên

3.1.8. Thiết kế bốn cách đặt vé đồng thời

Ở chức năng nhiều khách đặt vé, mỗi khách là một luồng với kết nối riêng; mọi luồng được giữ ở “vạch xuất phát” cho tới khi tất cả đã sẵn sàng rồi mới cùng chạy, để tạo tranh chấp thật. Giữa bước đọc và bước ghi có một khoảng “cân nhắc” (mặc định 100 ms) mô phỏng thời gian khách xem lại đơn hàng – đây chính là khe hở nơi tranh chấp xảy ra. Bốn cách xử lý được cài đặt như sau.

Cách 1 – Không dùng khóa (đọc rồi ghi đè):

```

START TRANSACTION;
SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1;           -- đọc được n
-- ... khách cân nhắc ...
UPDATE chuyen_bay SET so_ghe_trong = n - 1      -- ghi đè giá trị tự tính
WHERE ma_chuyen = 1;
INSERT INTO ve (ma_chuyen, hanh_khach) VALUES (1, 'Khách i');
COMMIT;
  
```

Cách 2 – Khóa bi quan: giống cách 1 nhưng câu đọc là `SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1 FOR UPDATE`, đặt khóa X lên dòng ngay khi đọc; các khách khác phải xếp hàng.

Cách 3 – Khóa lạc quan: mỗi lần thử là một giao tác mới; nếu câu `UPDATE` không tác động dòng nào (do version đã đổi) thì rollback, chờ ngẫu nhiên một chút rồi thử lại, tối đa 8 lần.

```
START TRANSACTION;
SELECT so_ghe_trong, version                -- đọc được n, v
  FROM chuyen_bay WHERE ma_chuyen = 1;
-- ... khách cần nhắc ...
UPDATE chuyen_bay SET so_ghe_trong = n - 1, version = version + 1
  WHERE ma_chuyen = 1 AND version = v;
-- 0 dòng bị tác động => có người ghi trước => ROLLBACK rồi thử lại
INSERT INTO ve (ma_chuyen, hanh_khach) VALUES (1, 'Khách i');
COMMIT;
```

Cách 4 – UPDATE nguyên tử có điều kiện: điều kiện còn ghế nằm ngay trong câu `UPDATE` nên không có khe hở giữa đọc và ghi; nếu 0 dòng bị tác động nghĩa là đã hết ghế.

```
START TRANSACTION;
UPDATE chuyen_bay SET so_ghe_trong = so_ghe_trong - 1
  WHERE ma_chuyen = 1 AND so_ghe_trong > 0;
INSERT INTO ve (ma_chuyen, hanh_khach) VALUES (1, 'Khách i');
COMMIT;
```

Ngoài ra, chế độ **deadlock** cho các khách giữ hai ghế 12A, 12B theo thứ tự ngược nhau (một nửa số khách giữ 12A trước, nửa còn lại giữ 12B trước) để quan sát InnoDB phát hiện và phá vòng chờ.

3.1.9. Giám sát khóa và các vấn đề gặp phải

Trong lúc kịch bản nhiều khách chạy, một luồng riêng lấy mẫu `data_locks` và `data_lock_waits` mỗi 25 ms để biết câu lệnh nào bị chặn và **bị ai chặn**. Quá trình xây dựng cho thấy một số “bẫy” khi giám sát khóa mà tài liệu ít đề cập, cần xử lý để số liệu đúng:

- `information_schema.innodb_trx` được phục vụ từ một bộ đệm chỉ làm mới khi đã hơn 0,1 giây không ai đọc; đọc liên tục sẽ mãi nhận ảnh chụp cũ. Ứng dụng dùng `performance_schema.threads` thay thế.

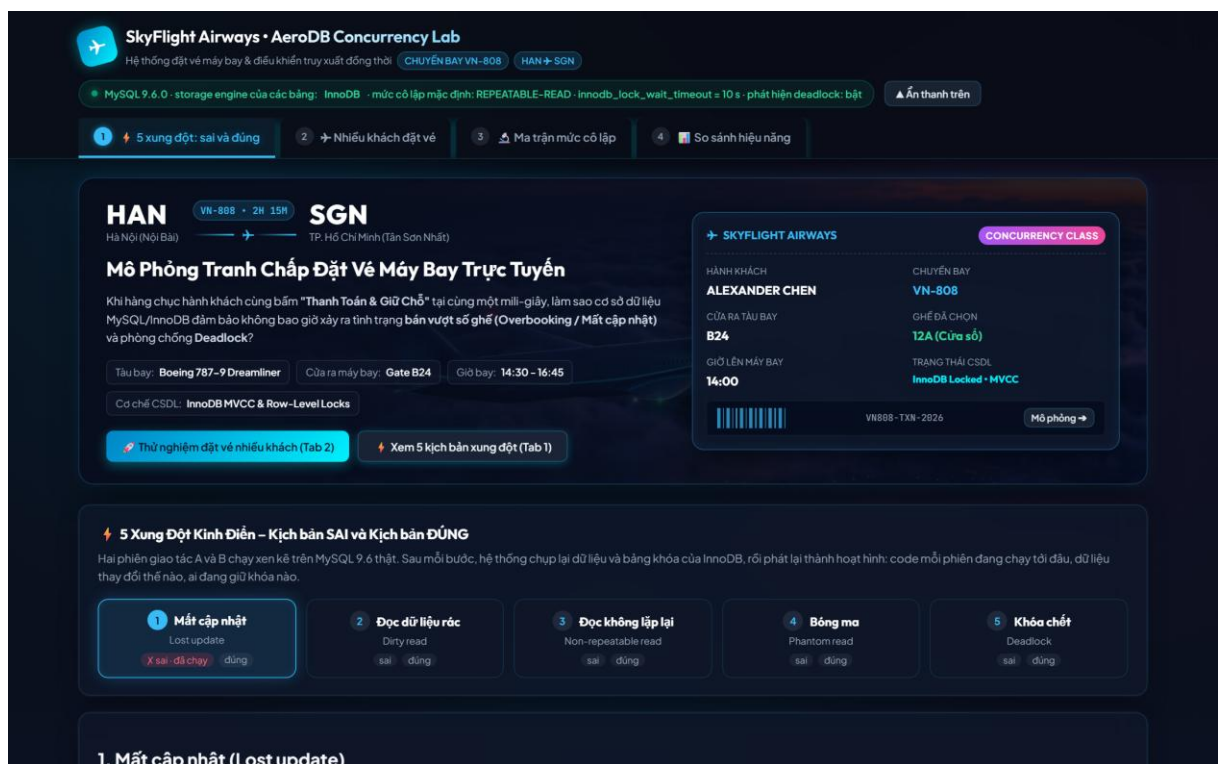
- `data_lock_waits` liệt kê cả những yêu cầu đang xếp hàng phía trước chứ không chỉ người đang giữ khóa; phải ghép với `data_locks` và lọc `LOCK_STATUS = 'GRANTED'` để biết ai thật sự chặn.
- `THREAD_ID` trong `data_locks` là luồng đã tạo cấu trúc khóa, chưa chắc là chủ sở hữu: khi B bị hủy vì khóa chết, khóa được trao cho A lại mang `THREAD_ID` của B. Chủ sở hữu thật được xác định qua `ENGINE_TRANSACTION_ID` (ứng dụng dùng khóa ý định mức bảng của mỗi giao tác để ánh xạ).
- Một truy vấn trên `performance_schema` không phải ảnh chụp nguyên tử; nếu chạy trùng lúc khóa đang được chuyển giao, có thể thấy cả người giữ cũ lẫn người giữ mới.

3.2. Xây dựng giao diện sản phẩm

Giao diện được tổ chức thành bốn tab tương ứng với các nhóm chức năng ở Hình 3.1. Các hình trong mục này là ảnh chụp trực tiếp từ ứng dụng khi chạy trên MySQL; số liệu trong hình và trong các bảng là kết quả của cùng một lần chạy.

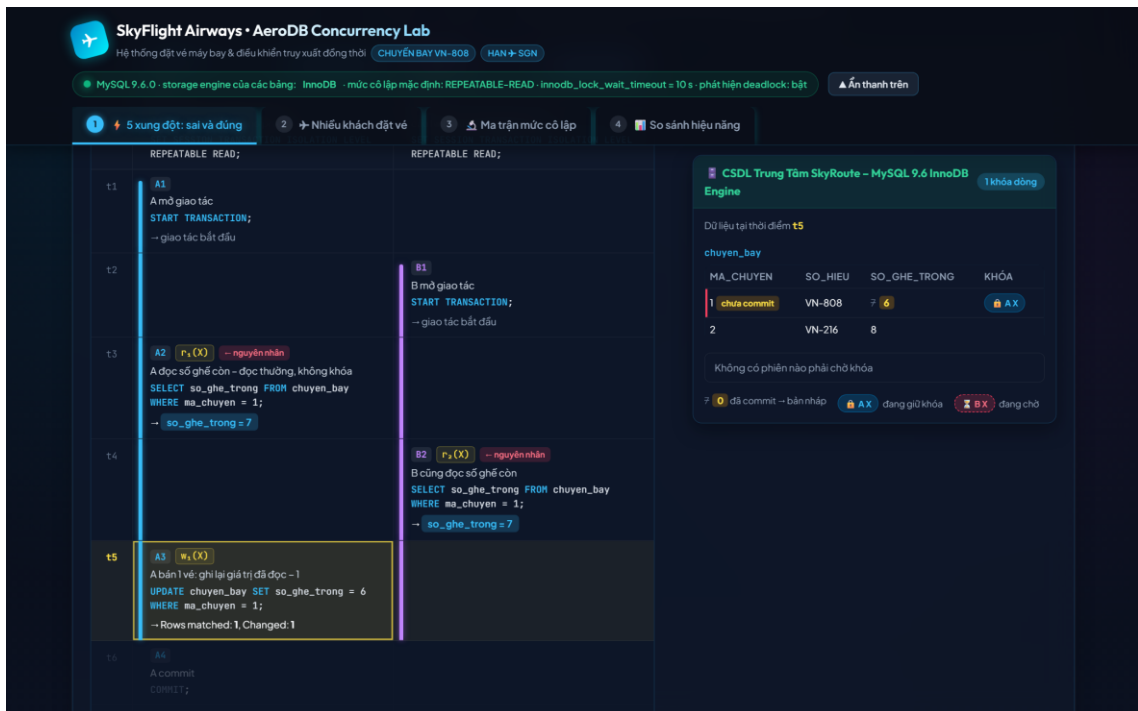
3.2.1. Giao diện tổng quan và điều hướng

Tên chức năng: màn hình chính. **Chức năng:** giới thiệu bối cảnh đặt vé chuyến VN-808 Hà Nội – TP. Hồ Chí Minh, hiển thị thông tin máy chủ (MySQL 9.6.0, engine InnoDB, mức cô lập mặc định, thời gian chờ khóa, trạng thái phát hiện khóa chết) và thanh tab để chuyển giữa bốn chức năng (Hình 3.5).

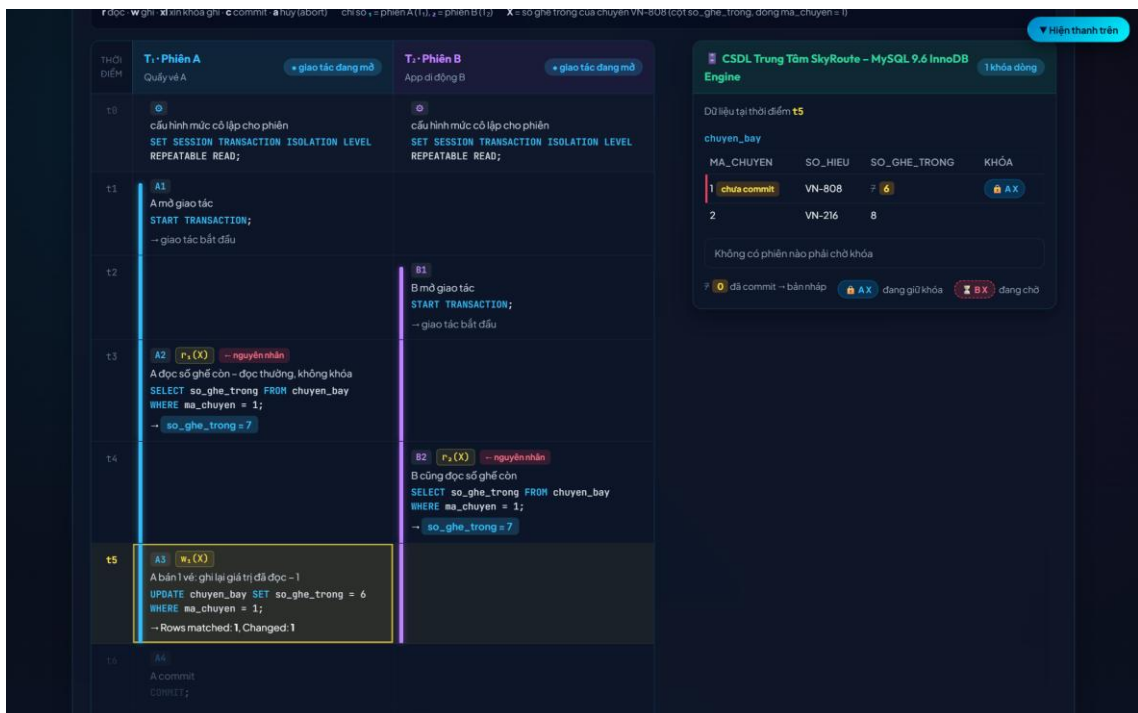


Hình 3.5. Giao diện tổng quan của ứng dụng

Thanh tiêu đề dính ở đầu màn hình nên khi cuộn xuống xem lịch giao tác, nó che mất một phần nội dung. Nút ▲ **Ẩn thanh trên** cho phép ẩn thanh này; nút nỏ ▼ **Hiện thanh trên** ở góc phải dùng để hiện lại, và lựa chọn được nhớ trong trình duyệt (Hình 3.6, Hình 3.7).



Hình 3.6. Khi thanh trên đang hiện, một phân lịch giao tác bị che



Hình 3.7. Sau khi bấm “Ấn thanh trên”, lịch giao tác và bảng CSDL hiện đầy đủ

3.2.2. Chức năng năm xung đột: sai và đúng

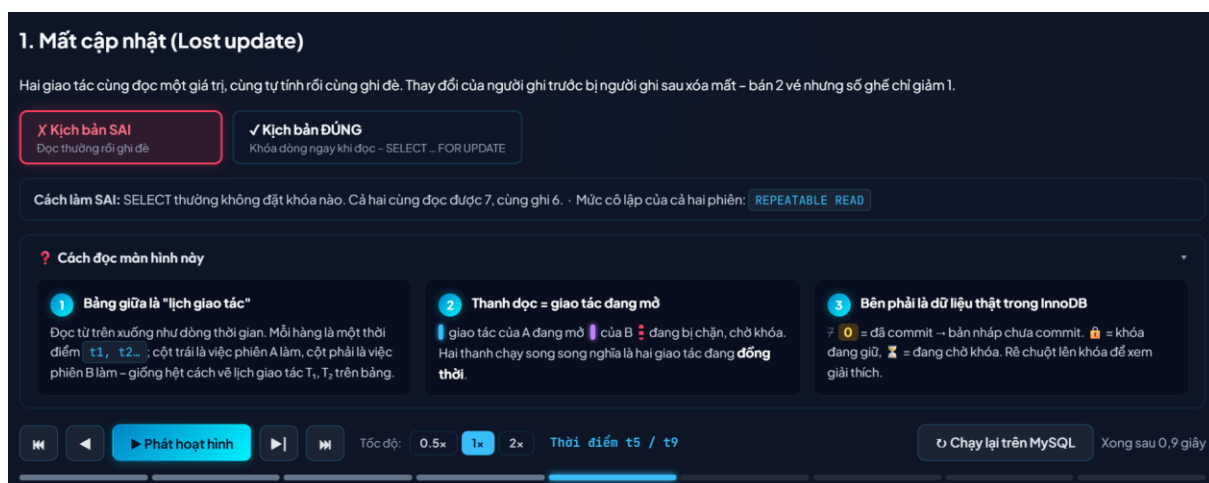
Tên chức năng: 5 xung đột: sai và đúng (tab 1). **Chức năng:** người dùng chọn một xung đột (Hình 3.8), chọn kịch bản SAI hoặc ĐÚNG; ứng dụng chạy kịch bản trên MySQL

rồi phát lại dưới dạng hoạt hình. Thanh điều khiển (Hình 3.9) cho phép phát/dừng, đi từng bước (phím $\leftarrow \rightarrow$), nhảy tới bước bất kỳ trên thanh tiến độ, chọn tốc độ 0,5× / 1× / 2× và chạy lại kịch bản trên MySQL.

Màn hình phát lại gồm: dòng tóm tắt bước hiện tại; khung “**Chuyện gì vừa xảy ra?**” giải thích bằng lời; **lịch thực thi** viết bằng ký hiệu giáo trình theo đúng thứ tự MySQL đã thực hiện; bảng **lịch giao tác** hai cột T₁ (phiên A – quầy vé) và T₂ (phiên B – ứng dụng di động), mỗi hàng là một thời điểm, kết quả hiện ngay dưới câu lệnh; thanh dọc cạnh mỗi cột thể hiện giao tác đang mở (liền màu) hoặc đang bị chặn chờ khóa (đỏ, đứt đoạn); và khung CSDL bên phải hiển thị dữ liệu cùng cột khóa tại thời điểm đang xem.



Hình 3.8. Danh sách năm xung đột



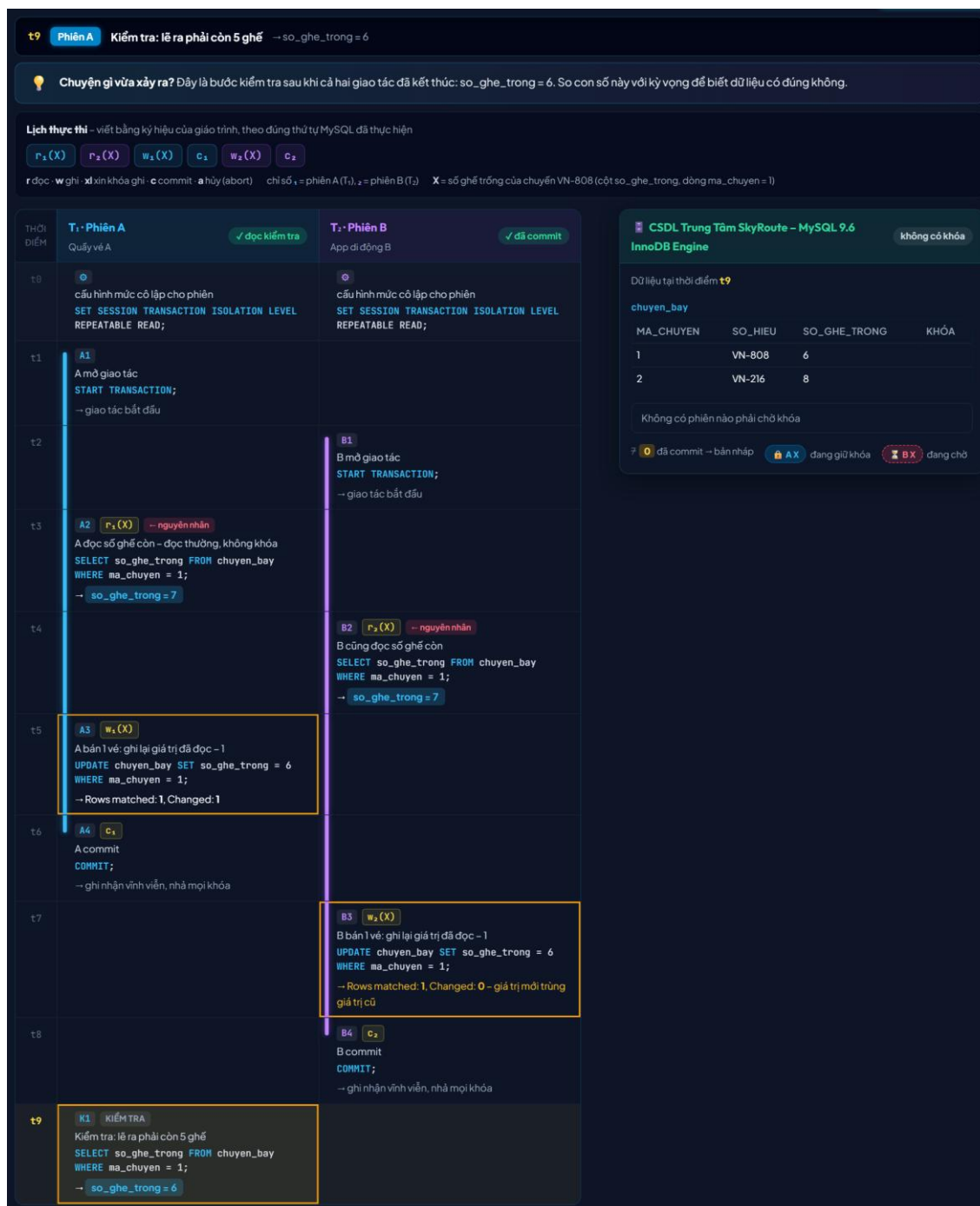
Hình 3.9. Chọn kịch bản, hướng dẫn đọc màn hình và thanh điều khiển phát lại

3.2.3. Xung đột 1: Mất cập nhật (Lost update)

Hai phiên cùng bán một vé của chuyến VN-808 khi còn 7 ghế trống. Kết quả đúng sau hai lần bán phải là 5.

Kịch bản SAI – Đọc thường rồi ghi đè (mức cô lập REPEATABLE READ). Lịch thực thi đo được: $r_1(X)$ $r_2(X)$ $w_1(X)$ c_1 $w_2(X)$ c_2 . Cả hai phiên đọc được 7 vì câu SELECT thường là đọc nhất quán, không đặt khóa. A ghi 6 và commit; B cũng ghi 6 – đúng giá trị A vừa ghi – nên MySQL chỉ trả về Rows matched: 1, Changed: 0, không có lỗi

hay cảnh báo nào. Bước kiểm tra cuối cho thấy còn 6 ghế (Hình 3.10). Điều này chứng minh mức REPEATABLE READ của MySQL **không** ngăn được mất cập nhật.

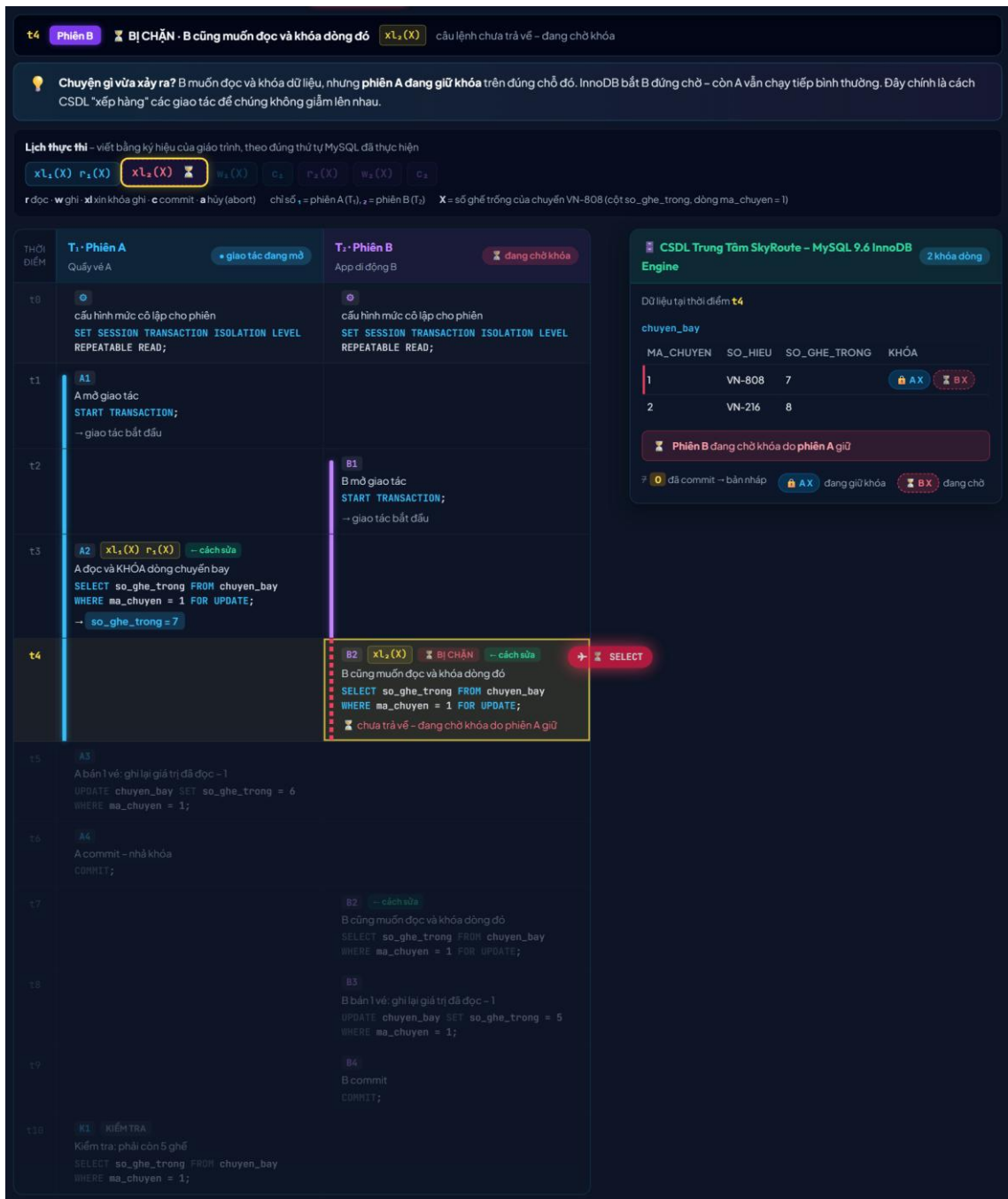


Hình 3.10. Mất cập nhật – kịch bản SAI tại bước then chốt

Kết luận do ứng dụng sinh ra từ kết quả thật của MySQL: “Cả A và B đều đọc thấy 7 ghế, cùng bán 1 vé và cùng commit thành công – tổng cộng 2 vé. Nhưng số ghế cuối cùng là 6, lẽ ra phải là 5. B đã ghi đè lên kết quả của A: đây là mất cập nhật. Tệ hơn nữa, câu

UPDATE của B chỉ nhận về 'Rows matched: 1, Changed: 0' vì nó ghi đúng giá trị 6 mà A đã ghi – ứng dụng không hề nhận được lỗi hay cảnh báo nào. REPEATABLE READ của MySQL KHÔNG ngăn được hiện tượng này, vì UPDATE luôn ghi lên bản mới nhất mà không kiểm tra dòng đó đã bị sửa sau snapshot hay chưa.”

Kịch bản ĐÚNG – Khóa dòng ngay khi đọc – SELECT ... FOR UPDATE (mức cô lập REPEATABLE READ). Lịch thực thi đo được: $x1_1(X)$ $r_1(X)$ $x1_2(X)$ $w_1(X)$ c_1 $r_2(X)$ $w_2(X)$ c_2 . Câu đọc của A dùng FOR UPDATE nên đặt khóa X lên dòng ngay khi đọc. Câu đọc của B cũng xin khóa X nên bị chặn (ký hiệu $x1_2(X)$ kèm đồng hồ cát) cho tới khi A commit (Hình 3.11); khi được chạy tiếp, B đọc giá trị **mới** là 6 và ghi 5. Đây là đọc hiện hành: dù ở REPEATABLE READ, câu đọc có khóa vẫn thấy bản mới nhất.



Hình 3.11. Mất cập nhật – kịch bản ĐỪNG tại bước then chốt

Kết luận của ứng dụng: “A khóa dòng ngay lúc đọc bằng FOR UPDATE. Câu đọc của B cũng xin khóa X nên bị chặn cho tới khi A commit; khi được chạy tiếp, B đọc được giá trị MỚI 6 (không phải 7) và ghi 5. Hai vé bán ra, số ghế giảm đúng 2: $7 \rightarrow 5$.”

Khi đã chạy cả hai kịch bản, thẻ **So sánh hai kịch bản** đặt chúng cạnh nhau (Hình 3.12).

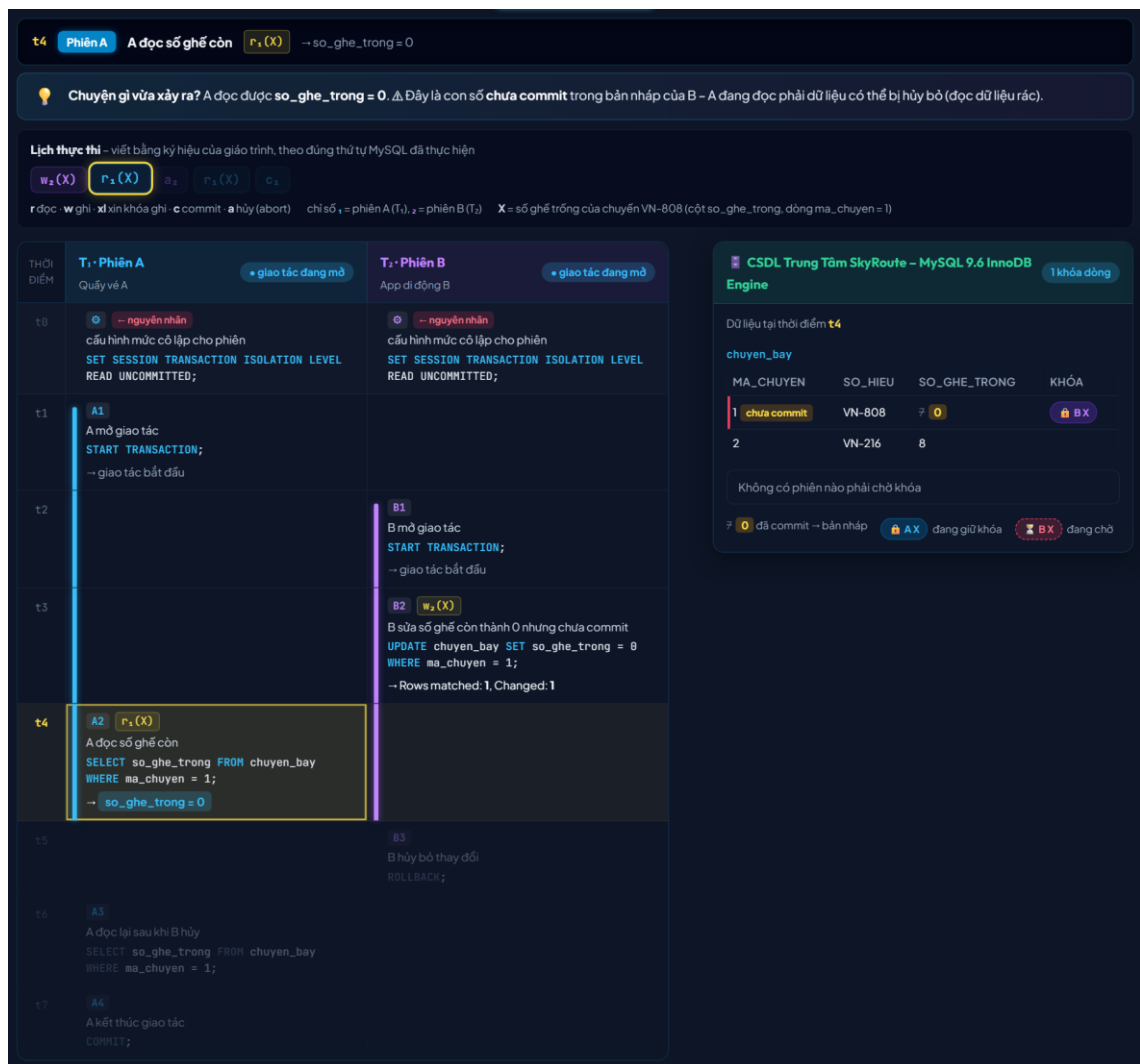
So sánh hai kịch bản	
X SAI: Đọc thường rồi ghi đè Mức cô lập: REPEATABLE READ Câu lệnh then chốt: A: SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1; B: SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1; Lịch thực thi: $r_2(X)$ $r_1(X)$ $w_1(X)$ c_1 $w_2(X)$ c_2 Kết quả: xung đột xảy ra Số lần bị chặn: 0 Thời gian chờ khóa: 0 ms Lỗi từ MySQL: không	✓ ĐÚNG: Khóa đồng ngay khi đọc – SELECT ... FOR UPDATE Mức cô lập: REPEATABLE READ Câu lệnh then chốt: A: SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1 FOR UPDATE; B: SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1 FOR UPDATE; Lịch thực thi: $x_{l_1}(X)$ $r_1(X)$ $x_{l_2}(X)$ $w_1(X)$ c_1 $r_2(X)$ $w_2(X)$ c_2 Kết quả: không xảy ra xung đột Số lần bị chặn: 1 Thời gian chờ khóa: 1.028 ms Lỗi từ MySQL: không
Khác biệt duy nhất: Chỉ thêm FOR UPDATE vào câu SELECT, mức cô lập giữ nguyên REPEATABLE READ. Điều này cho thấy mức cô lập mặc định của MySQL không tự chống được mất cập nhật.	

Hình 3.12. Mất cập nhật – so sánh hai kịch bản

3.2.4. Xung đột 2: Đọc dữ liệu rác (Dirty read)

Phiên B sửa số ghế trống thành 0 nhưng chưa commit (chẳng hạn đang giữ chỗ tạm), phiên A đọc số ghế, sau đó B rollback.

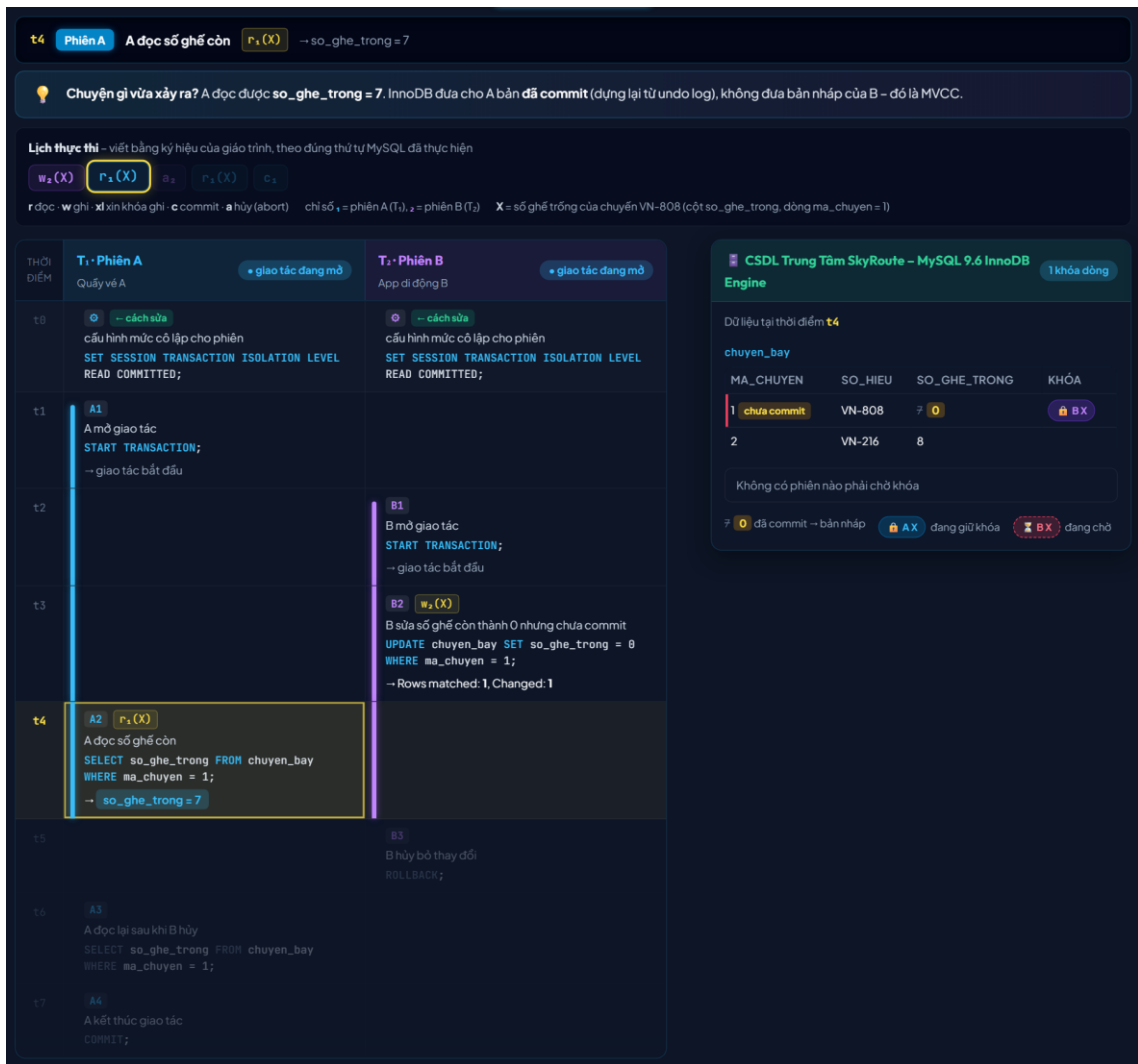
Kịch bản SAI – READ UNCOMMITTED (mức cô lập READ UNCOMMITTED). Lịch thực thi đo được: $w_2(X)$ $r_1(X)$ a_2 $r_1(X)$ c_1 . Ở READ UNCOMMITTED, câu SELECT của A đọc thẳng bản mới nhất trong InnoDB nên thấy 0 – con số chưa từng được commit (Hình 3.13). Trong khung CSDL, giá trị 0 được tô vàng kèm nhãn “chưa commit”.



Hình 3.13. Đọc dữ liệu rác – kịch bản SAI tại bước then chốt

Kết luận do ứng dụng sinh ra từ kết quả thật của MySQL: “A đọc được 0 – con số B vừa ghi nhưng CHƯA commit. Sau đó B rollback, nên giá trị thật vẫn là 7. A đã dựa vào một dữ liệu chưa từng tồn tại chính thức: đó là đọc dữ liệu rác.”

Kịch bản ĐÚNG – READ COMMITTED (mức cô lập READ COMMITTED). Lịch thực thi đo được: $w_2(X)$ $r_1(X)$ a_2 $r_1(X)$ c_1 . Chỉ đổi mức cô lập lên READ COMMITTED, câu lệnh giữ nguyên. A đọc được 7 – bản đã commit – do InnoDB dựng lại từ undo log, và A **không phải chờ** B (Hình 3.14). Đây là ưu điểm của MVCC so với cài đặt bằng khóa thuần túy.



Hình 3.14. Đọc dữ liệu rác – kịch bản ĐÚNG tại bước then chốt

Kết luận của ứng dụng: “A đọc được 7 – bản đã commit gần nhất. InnoDB dừng lại bản cũ từ undo log (MVCC) nên A không thấy thay đổi dở dang của B, và cũng không phải chờ.”

Khi đã chạy cả hai kịch bản, thẻ **So sánh hai kịch bản** đặt chúng cạnh nhau (Hình 3.15).

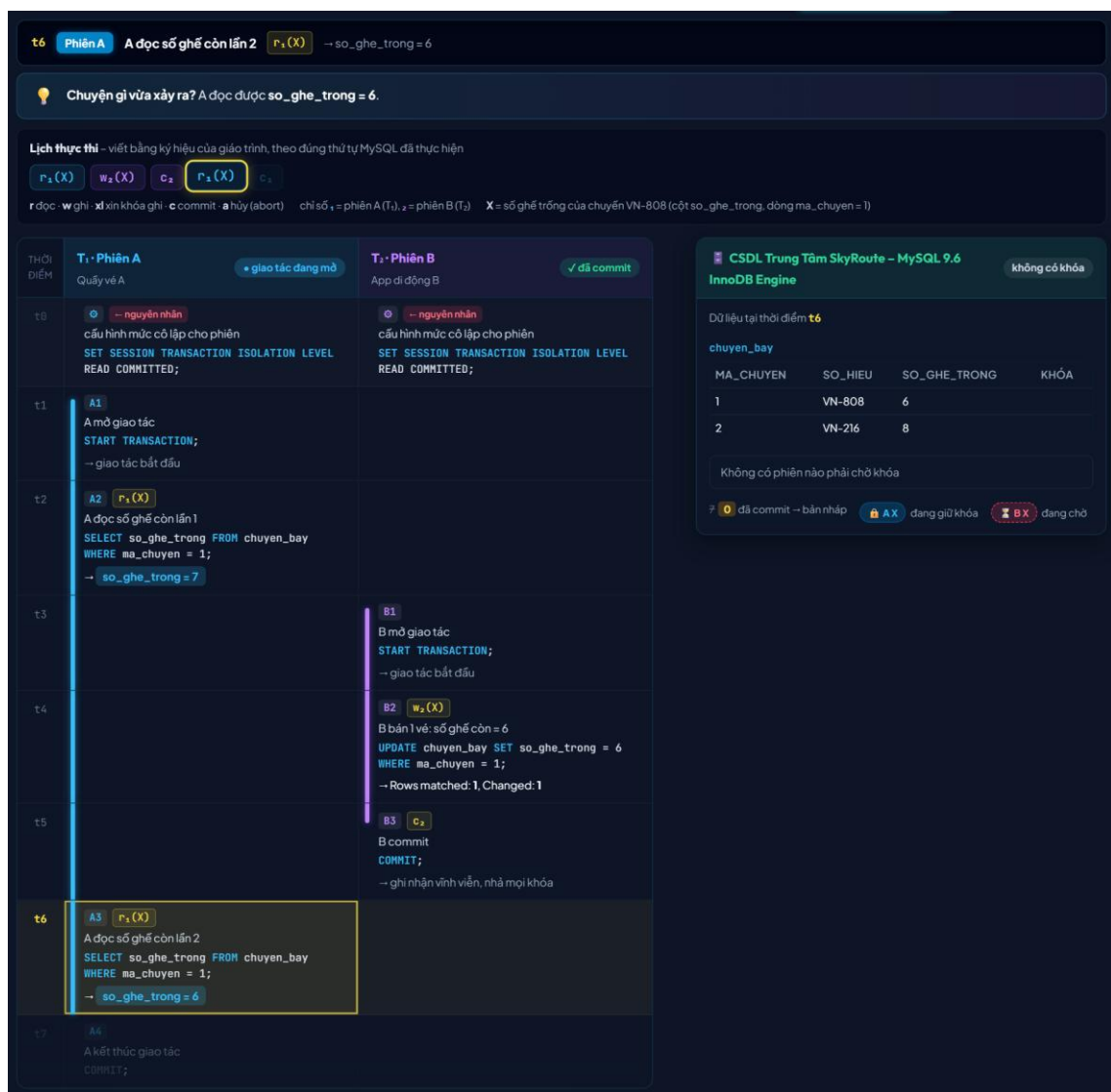
So sánh hai kịch bản	
✗ SAI: READ UNCOMMITTED Mức cô lập: <code>READ_UNCOMMITTED</code> Câu lệnh then chốt: <code>A: SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;</code> Lịch thực thi: <code>w₂(X) r₁(X) a₂ r₂(X) c₁</code> Kết quả: xung đột xảy ra Số lần bị chặn: 0 Thời gian chờ khóa: 0 ms Lỗi từ MySQL: không	✓ ĐÚNG: READ COMMITTED Mức cô lập: <code>READ_COMMITTED</code> Câu lệnh then chốt: <code>A: SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;</code> Lịch thực thi: <code>w₂(X) r₁(X) a₂ r₂(X) c₁</code> Kết quả: không xảy ra xung đột Số lần bị chặn: 0 Thời gian chờ khóa: 0 ms Lỗi từ MySQL: không
Khắc biệt duy nhất: Câu lệnh giống hệt nhau, chỉ đổi mức cô lập từ READ UNCOMMITTED lên READ COMMITTED. InnoDB dựng lại bản đã commit từ undo log (MVCC) nên A không phải chờ B.	

Hình 3.15. Đọc dữ liệu rác – so sánh hai kịch bản

3.2.5. Xung đột 3: Đọc không lặp lại (Non-repeatable read)

Phiên A đọc số ghế trống hai lần trong cùng một giao tác; giữa hai lần đọc, phiên B bán một vé (ghi 6) và commit.

Kịch bản SAI – READ COMMITTED (mức cô lập READ COMMITTED). Lịch thực thi đo được: $r_1(X) \ w_2(X) \ c_2 \ r_1(X) \ c_1$. Ở READ COMMITTED, mỗi câu SELECT lấy một snapshot mới nên lần đọc thứ hai của A thấy 6 trong khi lần đầu thấy 7 (Hình 3.16).



Hình 3.16. Đọc không lặp lại – kịch bản SAI tại bước then chốt

Kết luận do ứng dụng sinh ra từ kết quả thật của MySQL: “Trong CÙNG một giao tác, A đọc cùng một dòng hai lần nhưng ra 7 rồi 6, vì giữa hai lần đọc B đã sửa và commit. Mỗi câu SELECT ở mức này lấy một snapshot mới.”

Kịch bản ĐÚNG – REPEATABLE READ (mức cô lập REPEATABLE READ). Lịch thực thi đo được: $r_1(X)$ $w_2(X)$ c_2 $r_1(X)$ c_1 . Ở REPEATABLE READ, snapshot được tạo ở lần đọc đầu tiên và dùng lại suốt giao tác, nên A đọc hai lần đều ra 7, trong khi B vẫn sửa và commit bình thường, không ai bị chặn (Hình 3.17).

t6 Phiên A A đọc số ghế còn lần 2 **r₁(X)** → so_ghe_trong = 7

Chuyện gì vừa xảy ra? A đọc được so_ghe_trong = 7. Dù dữ liệu đã commit giờ là 6, A vẫn thấy 7: A đọc trên **snapshot** chụp từ lần đọc đầu tiên của giao tác (MVCC).

Lịch thực thi - viết bằng ký hiệu của giao trình, theo đúng thứ tự MySQL đã thực hiện

r đọc **w** ghi **x** xin khóa ghi **c** commit **a** hủy (abort) chỉ số ₁ = phiên A (T₁), ₂ = phiên B (T₂) **X** = số ghế trống của chuyến VN-808 (cột so_ghe_trong, dòng ma_chuyen = 1)

THỜI ĐIỂM	T ₁ - Phiên A Quầy vé A + giao tác đang mở	T ₂ - Phiên B App di động B ✓ đã commit
t0	o → cách sửa cấu hình mức cô lập cho phiên SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;	o → cách sửa cấu hình mức cô lập cho phiên SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;
t1	A1 A mở giao tác START TRANSACTION; → giao tác bắt đầu	
t2	A2 r₁(X) A đọc số ghế còn lần 1 SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1; → so_ghe_trong = 7	
t3		B1 B mở giao tác START TRANSACTION; → giao tác bắt đầu
t4		B2 w₂(X) B bán 1 vé: số ghế còn = 6 UPDATE chuyen_bay SET so_ghe_trong = 6 WHERE ma_chuyen = 1; → Rows matched: 1, Changed: 1
t5		B3 c₂ B commit COMMIT; → ghi nhận vĩnh viễn, nhả mọi khóa
t6	A3 r₁(X) A đọc số ghế còn lần 2 SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1; → so_ghe_trong = 7	
t7	A4 A kết thúc giao tác COMMIT;	

CSDL Trung Tâm SkyRoute - MySQL 9.6 InnoDB Engine **không có khóa**

Dữ liệu tại thời điểm **t6**

chuyen_bay			
MA_CHUYEN	SO_HIEU	SO_GHE_TRONG	KHÓA
1	VN-808	6	
2	VN-216	8	

Không có phiên nào phải chờ khóa

7 **o** đã commit → bán nhập **a** **AX** đang giữ khóa **B** **BX** đang chờ

Hình 3.17. Đọc không lặp lại – kịch bản ĐÚNG tại bước then chốt

Kết luận của ứng dụng: “A đọc hai lần đều ra 7, dù B đã commit giá trị mới. Ở *REPEATABLE READ*, snapshot được tạo ở lần đọc đầu tiên và dùng lại suốt giao tác (MVCC), B không hề bị chặn.”

Khi đã chạy cả hai kịch bản, thẻ **So sánh hai kịch bản** đặt chúng cạnh nhau (Hình 3.18).

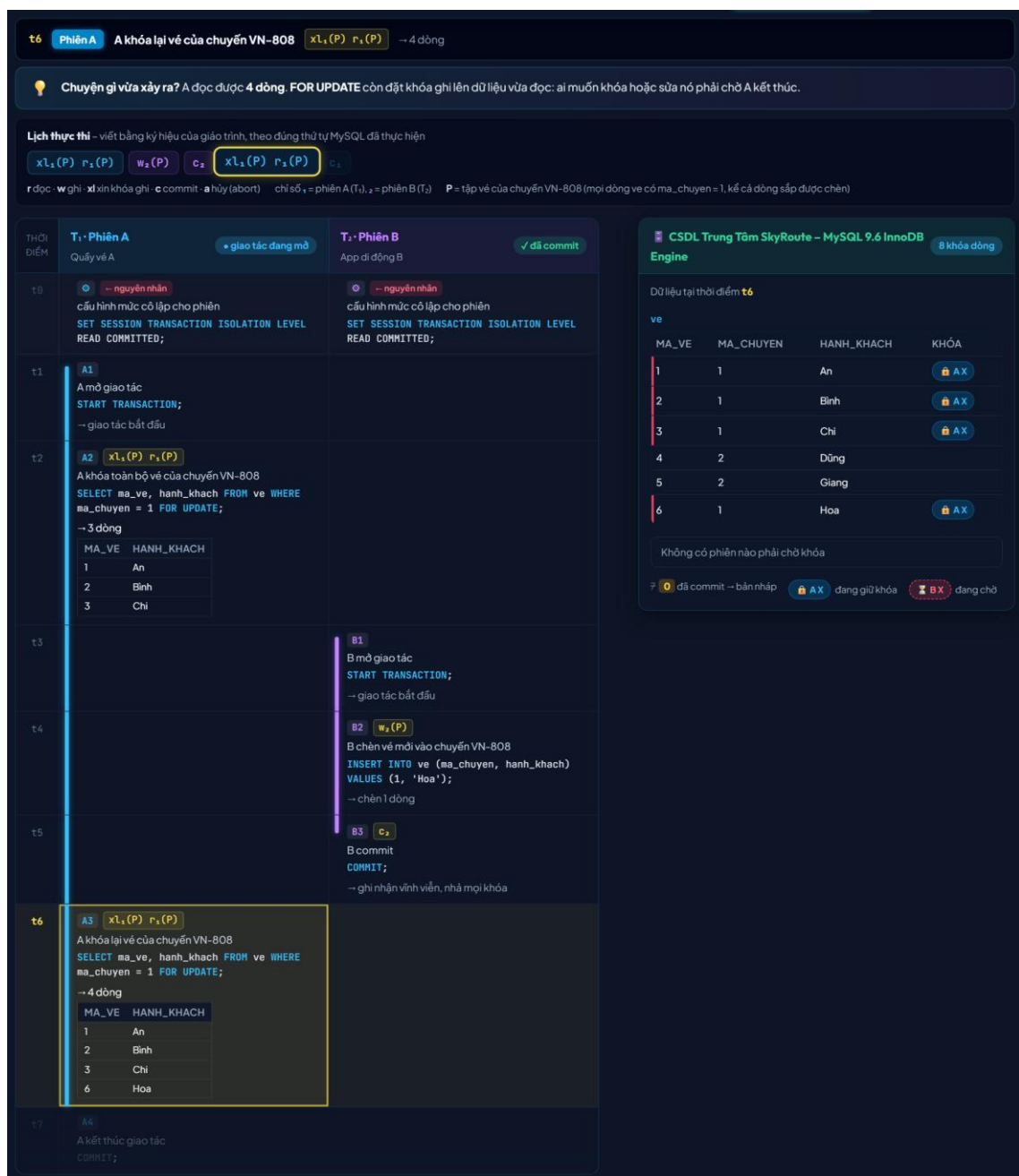
So sánh hai kịch bản	
✗ SAI: READ COMMITTED Mức cô lập: <code>READ COMMITTED</code> Câu lệnh then chốt: <code>A: SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;</code> Lịch thực thi: <code>r₁(X) w₂(X) c₂ r₁(X) c₁</code> Kết quả: xung đột xảy ra Số lần bị chặn: 0 Thời gian chờ khóa: 0 ms Lỗi từ MySQL: không	✓ ĐÚNG: REPEATABLE READ Mức cô lập: <code>REPEATABLE READ</code> Câu lệnh then chốt: <code>A: SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;</code> Lịch thực thi: <code>r₁(X) w₂(X) c₂ r₁(X) c₁</code> Kết quả: không xảy ra xung đột Số lần bị chặn: 0 Thời gian chờ khóa: 0 ms Lỗi từ MySQL: không
Khác biệt duy nhất: Câu lệnh giống hệt nhau, chỉ đổi từ READ COMMITTED lên REPEATABLE READ. A dùng lại một snapshot suốt giao tác, B vẫn sửa và commit bình thường, không ai phải chờ.	

Hình 3.18. Đọc không lặp lại – so sánh hai kịch bản

3.2.6. Xung đột 4: Bóng ma (Phantom read)

Phiên A khóa toàn bộ vé của chuyến VN-808 bằng `SELECT ... FOR UPDATE` (3 vé), phiên B cố chèn thêm vé cho đúng chuyến đó rồi commit, sau đó A khóa lại.

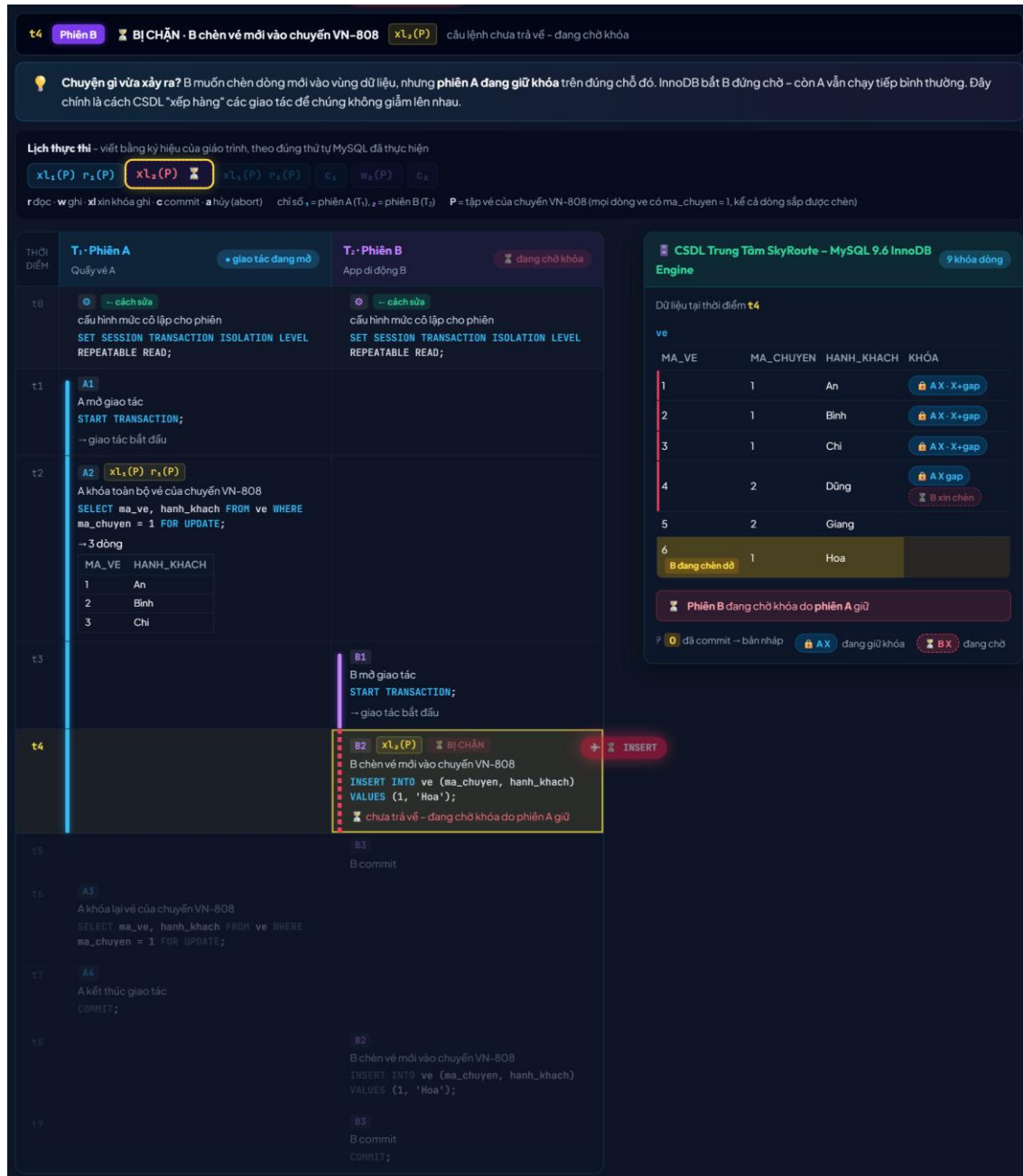
Kịch bản SAI – READ COMMITTED – chỉ khóa dòng (mức cô lập READ COMMITTED). Lịch thực thi đo được: $x1_1(P) \ r_1(P) \ w_2(P) \ c_2 \ x1_1(P) \ r_1(P) \ c_1$. Ở READ COMMITTED, InnoDB chỉ khóa 3 bản ghi đang tồn tại (record lock), khoảng trống giữa chúng vẫn mở; INSERT của B thành công và lần khóa thứ hai của A thấy 4 dòng – dòng thứ tư là “bóng ma” (Hình 3.19).



Kết luận do ứng dụng sinh ra từ kết quả thật của MySQL: “Lần khóa đầu A giữ 3 dòng, lần sau ra 4 dòng. Ở mức này InnoDB chỉ khóa các dòng đang tồn tại (record lock), KHÔNG khóa khoảng trống giữa chúng, nên B chen được dòng mới vào ngay trong vùng A tưởng đã khóa hết.”

Kịch bản ĐÚNG – REPEATABLE READ – khóa cả khoảng trống (mức cô lập REPEATABLE READ). Lịch thực thi đo được: $x1_1(P)$ $r_1(P)$ $x1_2(P)$ $x1_1(P)$ $r_1(P)$ c_1

$w_2(P) \subset c_2$. Ở REPEATABLE READ, FOR UPDATE đặt next-key lock: khóa cả bản ghi lẫn khoảng trống. INSERT của B rơi vào khoảng đã bị khóa nên phải chờ khóa ý định chèn và bị chặn cho tới khi A commit (Hình 3.20). Hai lần khóa của A đều thấy 3 dòng.



Hình 3.20. Bóng ma – kịch bản ĐÚNG tại bước then chốt

Kết luận của ứng dụng: “A khóa hai lần đều ra 3 dòng. Lệnh INSERT của B bị chặn bởi GAP LOCK: InnoDB khóa cả khoảng trống sau dòng cuối cùng thỏa $ma_chuyen = 1$ (next-key lock), nên B phải chờ tới khi A commit. Xem bảng khóa: B chờ khóa INSERT_INTENTION, xung đột với khóa GAP của A.”

Khi đã chạy cả hai kịch bản, thẻ **So sánh hai kịch bản** đặt chúng cạnh nhau (Hình 3.21).

So sánh hai kịch bản

✗ SAI: READ COMMITTED – chỉ khóa dòng

Mức cô lập: **READ COMMITTED**

Câu lệnh then chốt: **A: SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;**

Lịch thực thi: $x_{1_1}(P) \quad r_{1_1}(P) \quad w_2(P) \quad c_2 \quad x_{1_2}(P) \quad r_{1_2}(P) \quad c_1$

Kết quả: **xung đột xảy ra**

Số lần bị chặn: **0**

Thời gian chờ khóa: **0 ms**

Lỗi từ MySQL: **không**

✓ ĐÚNG: REPEATABLE READ – khóa cả khoảng trống

Mức cô lập: **REPEATABLE READ**

Câu lệnh then chốt: **A: SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;**

Lịch thực thi: $x_{1_1}(P) \quad r_{1_1}(P) \quad x_{1_2}(P) \quad x_{1_2}(P) \quad x_{1_1}(P) \quad r_{1_2}(P) \quad c_1 \quad w_2(P) \quad c_2$

Kết quả: **không xảy ra xung đột**

Số lần bị chặn: **1**

Thời gian chờ khóa: **1.024 ms**

Lỗi từ MySQL: **không**

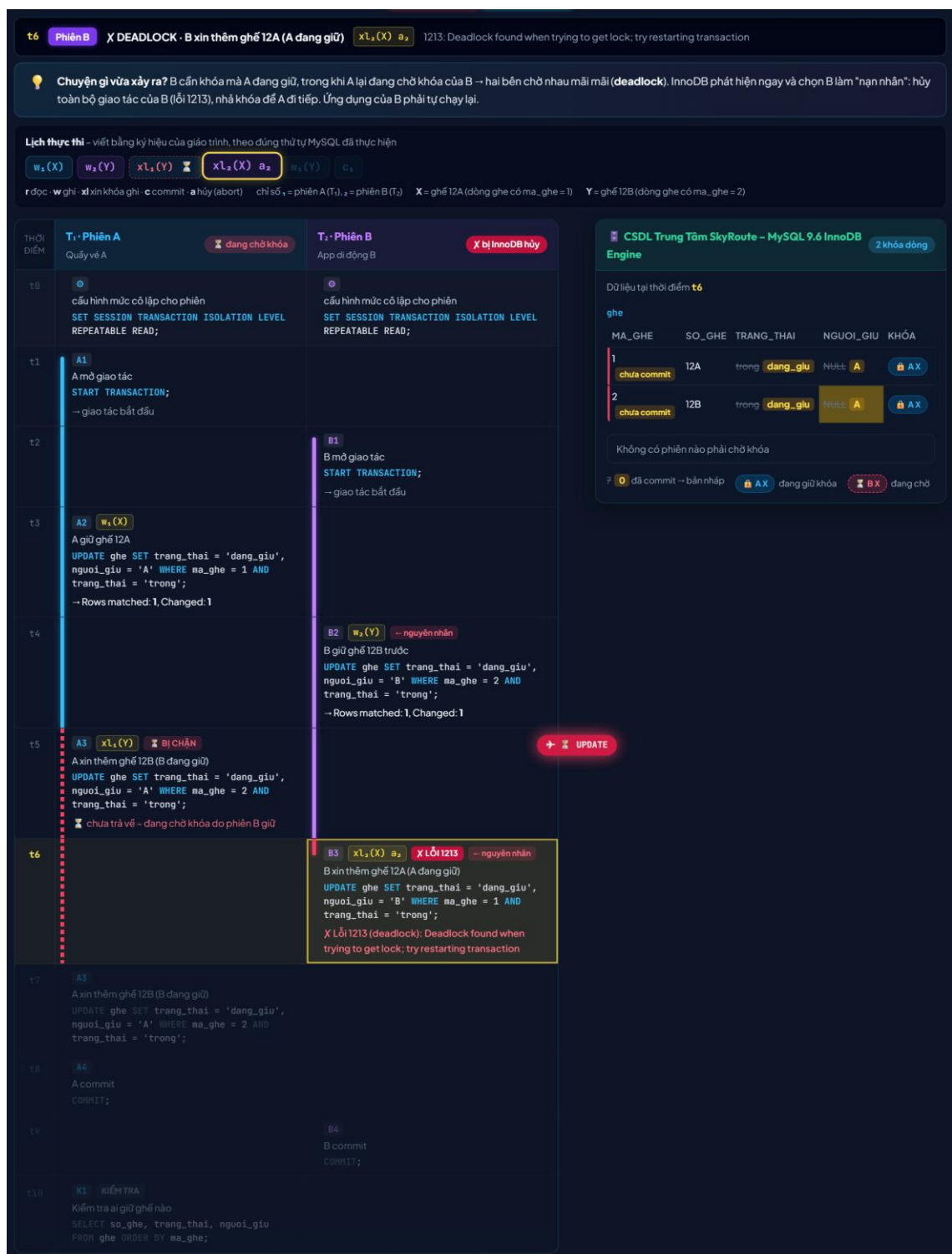
Khác biệt duy nhất: Câu lệnh giống hệt nhau. Ở READ COMMITTED, InnoDB chỉ khóa các dòng đang có; ở REPEATABLE READ nó khóa cả khoảng trống giữa các dòng (gap lock), nên B không chèn được.

Hình 3.21. Bóng ma – so sánh hai kịch bản

3.2.7. Xung đột 5: Khóa chết (Deadlock)

Hai phiên cùng muốn giữ hai ghế 12A và 12B. Mỗi lần giữ ghế là một câu UPDATE đặt khóa X lên dòng ghế.

Kịch bản SAI – Hai bên khóa ghế theo thứ tự ngược nhau (mức cô lập REPEATABLE READ). Lịch thực thi đo được: $w_1(X) \quad w_2(Y) \quad x_{1_1}(Y) \quad x_{1_2}(X) \quad a_2 \quad w_1(Y) \quad c_1$. A giữ 12A, B giữ 12B, rồi A xin 12B (bị chặn vì B đang giữ) và B xin 12A: hai bên chờ nhau thành vòng tròn. InnoDB phát hiện ngay lập tức, không đợi hết thời gian chờ khóa, chọn B làm nạn nhân và trả về lỗi **1213** (Hình 3.22). Ký hiệu $x_{1_2}(\dots) \quad a_2$ cho thấy B xin khóa rồi bị hủy; A được chạy tiếp và giữ cả hai ghế.

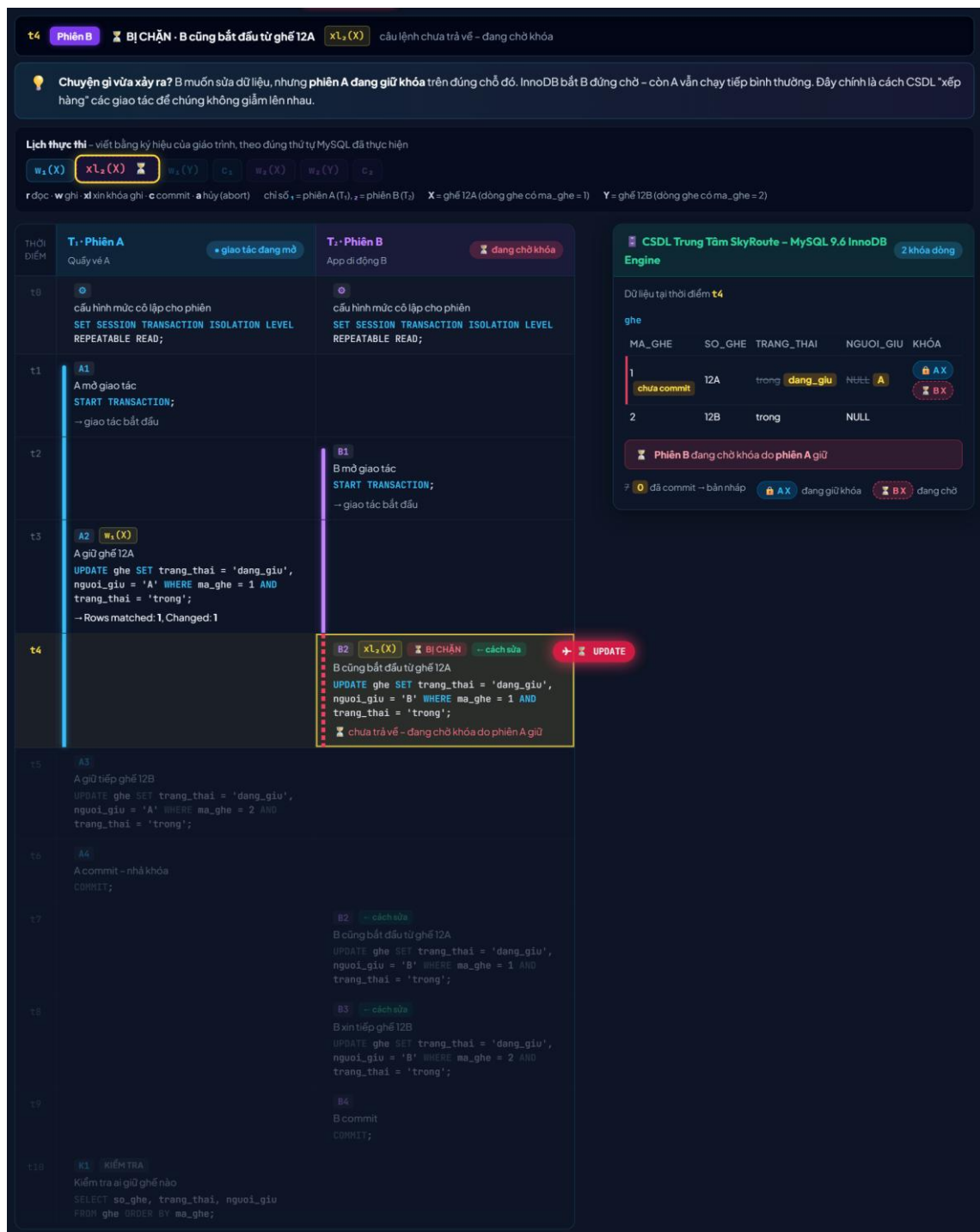


Hình 3.22. Khóa chết – kịch bản SAI tại bước then chốt

Kết luận do ứng dụng sinh ra từ kết quả thật của MySQL: “A giữ ghế 12A rồi xin ghế 12B, còn B giữ ghế 12B rồi xin ghế 12A: mỗi bên đang giữ đúng thứ bên kia cần, hai bên chờ nhau thành vòng tròn. InnoDB phát hiện vòng chờ NGAY (không đợi hết innodb_lock_wait_timeout), chọn B làm nạn nhân và ROLLBACK toàn bộ giao tác của B

với lỗi 1213. Nhờ vậy A chạy tiếp được. Kết quả cuối: ghế 12A – A, ghế 12B – A. Ứng dụng phía B bắt buộc phải bắt lỗi và chạy lại giao tác.”

Kịch bản ĐÚNG – Mọi giao tác khóa ghế theo cùng một thứ tự (mức cô lập REPEATABLE READ). Lịch thực thi đo được: $w_1(X) \rightarrow l_2(X) \rightarrow w_1(Y) \rightarrow c_1 \rightarrow w_2(X) \rightarrow w_2(Y) \rightarrow c_2$. Chỉ đổi thứ tự khóa của B: cả hai đều khóa 12A trước. B phải chờ A ở ghế 12A, nhưng A không cần thứ gì B đang giữ nên **không có vòng chờ** – chỉ là xếp hàng bình thường (Hình 3.23). Khi A commit, B đọc lại bản mới nhất, thấy ghế đã có người giữ (Rows matched: 0) nên không giữ được ghế nào – đúng nghiệp vụ và không có lỗi.



Hình 3.23. Khóa chết – kịch bản ĐÚNG tại bước then chốt

Kết luận của ứng dụng: “Cả A và B đều xin ghé 12A trước rồi mới tới ghé 12B. B phải chờ A ở ghé 12A, nhưng A không cần thứ gì B đang giữ nên KHÔNG có vòng chờ – chỉ là xếp hàng bình thường. A giữ xong cả hai ghé và commit; B được chạy tiếp, đọc lại dòng mới nhất và thấy ghé đã có người giữ (Rows matched: 0) nên không giữ được ghé nào – đúng nghiệp vụ, không có lỗi. Kết quả cuối: ghé 12A – A, ghé 12B – A.”

Khi đã chạy cả hai kịch bản, thẻ **So sánh hai kịch bản** đặt chúng cạnh nhau (Hình 3.24).

So sánh hai kịch bản

X SAI: Hai bên khóa ghế theo thứ tự ngược nhau

Mức cô lập: REPEATABLE READ

Câu lệnh then chốt: B: UPDATE ghe SET trang_thai = 'dang_giu', nguoi_giu = 'B' WHERE ma_ghe = 2 AND trang_thai = 'trong';
B: UPDATE ghe SET trang_thai = 'dang_giu', nguoi_giu = 'B' WHERE ma_ghe = 1 AND trang_thai = 'trong';

Lịch thực thi: w₁(X) w₂(Y) x₁(Y) x₂(X) a₂ w₁(Y) c₁

Kết quả: xung đột xảy ra

Số lần bị chặn: 1

Thời gian chờ khóa: 711ms

Lỗi từ MySQL: 1213 deadlock

✓ ĐÚNG: Mọi giao tác khóa ghế theo cùng một thứ tự

Mức cô lập: REPEATABLE READ

Câu lệnh then chốt: B: UPDATE ghe SET trang_thai = 'dang_giu', nguoi_giu = 'B' WHERE ma_ghe = 1 AND trang_thai = 'trong';
B: UPDATE ghe SET trang_thai = 'dang_giu', nguoi_giu = 'B' WHERE ma_ghe = 2 AND trang_thai = 'trong';

Lịch thực thi: w₁(X) x₁(X) w₁(Y) c₁ w₂(X) w₂(Y) c₂

Kết quả: không xảy ra xung đột

Số lần bị chặn: 1

Thời gian chờ khóa: 1.023ms

Lỗi từ MySQL: không

Khác biệt duy nhất: Chỉ đổi THỨ TỰ khóa của B: luôn khóa ghế có mã nhỏ trước. Khi mọi giao tác khóa tài nguyên theo cùng một thứ tự thì không thể hình thành vòng chờ.

Hình 3.24. Khóa chết – so sánh hai kịch bản

Bảng 3.6 tổng hợp kết quả đo được của mười kịch bản: mọi kịch bản SAI đều tái hiện được xung đột và mọi kịch bản ĐÚNG đều ngăn được nó, đúng như thiết kế.

Bảng 3.6. Kết quả đo được của năm cặp kịch bản

Xung đột	SAI: xảy ra	ĐÚNG: xảy ra	ĐÚNG: số lần bị chặn	ĐÚNG: thời gian chờ	Lỗi (SAI)	Lỗi (ĐÚNG)
Mất cập nhật	Có	Không	1	1.028 ms	—	—
Đọc dữ liệu rác	Có	Không	0	0	—	—
Đọc không lặp lại	Có	Không	0	0	—	—
Bóng ma	Có	Không	1	1.024 ms	—	—
Khóa chết	Có	Không	1	1.023 ms	1213 deadlock	—

3.2.8. Chức năng mô phỏng nhiều khách đặt vé

Tên chức năng: Nhiều khách đặt vé (tab 2). **Chức năng:** khác với tab 1 (thứ tự xen kẽ cố định), ở đây nhiều khách thật sự xuất phát cùng một mili giây và tranh nhau số ghế có hạn. Người dùng chọn cách xử lý tranh chấp, số khách (1–60), số ghế mở bán, thời gian cân nhắc và mức cô lập (Hình 3.25); khung bên dưới mô tả cách làm và mã SQL tương ứng.

Nhiều Khách Cùng Đặt Vé Máy Bay Chuyến VN-808

Mỗi khách hàng là một luồng (thread) với kết nối MySQL riêng. Tất cả cùng xuất phát đồng thời tại mili-giây thứ 0 để tranh nhau số ghế có hạn trên chuyến bay.

Phương pháp xử lý tranh chấp: Không dùng khóa (đọc rồi ghi đè) | Số hành khách đặt vé: 10 | Số ghế mở bán: 5 | Thời gian cân nhắc (ms): 100 | Mức cô lập: Mặc định (REPEATABLE READ) | **Chạy mô phỏng đặt vé**

Mỗi khách đọc số ghế còn, cân nhắc, rồi tự tính và ghi đè giá trị mới. Không có gì ngăn hai khách cùng đọc một con số – nhiều người sẽ cùng ghi đè cùng một giá trị: **mất cập nhật**.

```
START TRANSACTION;
SELECT so_ghe_trong FROM chuyen_bay WHERE ma_chuyen = 1; -- đọc được n
-- _ khách cân nhắc _
UPDATE chuyen_bay SET so_ghe_trong = n - 1 WHERE ma_chuyen = 1;
INSERT INTO ve (ma_chuyen, hanh_khach) VALUES (1, 'Khách i');
COMMIT;
```

Hình 3.25. Cấu hình mô phỏng nhiều khách đặt vé

Với **không dùng khóa**, 10 khách tranh 5 ghế: hệ thống bán ra 10 vé, **bán vượt 5 vé**, trong khi cột số ghế trống vẫn báo 4 (Hình 3.26). Biểu đồ diễn biến (Hình 3.27) cho thấy mỗi hàng là một giao tác: đoạn xám dài là lúc khách cân nhắc khi giao tác vẫn mở, đoạn đỏ là lúc bị chặn chờ khóa. Đáng chú ý là ngay cả khi không dùng khóa, vẫn có 8 lần chờ khóa vì câu UPDATE luôn phải xin khóa X; nhưng khóa đến quá muộn – sau khi giá trị đã được đọc. Sơ đồ khoang ghế đánh dấu các ghế bị bán trùng (Hình 3.28).

Kết quả mô phỏng

Số ghế ban đầu 5	Vé đã bán 10	Ghế còn lại 4	Bán vượt 5	Thủ lại 0	Lần chờ khóa 8	Thời gian 218 ms
----------------------------	------------------------	-------------------------	----------------------	---------------------	--------------------------	----------------------------

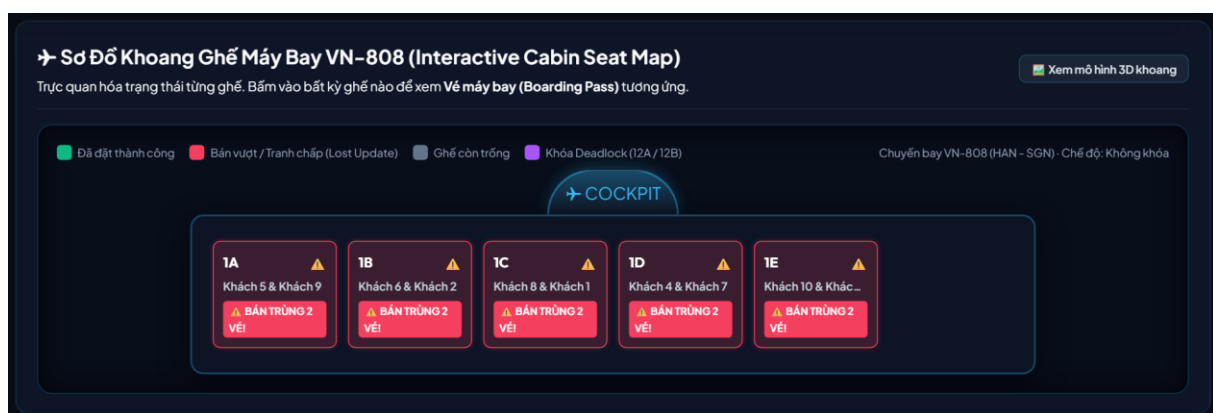
Dữ liệu đã sai. 10 khách tranh 5 ghế, hệ thống bán ra 10 vé – thừa 5 vé, trong khi cột số ghế còn lại vẫn báo 4. Nhiều giao tác cùng đọc được một con số rồi cùng ghi đè cùng một giá trị: **mất cập nhật (lost update)**. Xem nhật ký: nhiều khách cùng “đọc được so_ghe_trong = 5”.

Lưu ý: đây là mức REPEATABLE READ – mức mặc định của MySQL vẫn không ngăn được lỗi này.
Mức cô lập: mặc định (REPEATABLE READ) - cách xử lý: Không dùng khóa (đọc rồi ghi đè)

Hình 3.26. Kết quả khi không dùng khóa: bán vượt



Hình 3.27. Diễn biến bên trong CSDL khi không dùng khóa



Hình 3.28. Sơ đồ khoang ghế: các ghế bị bán trùng

Với **khóa bi quan**, kết quả đúng: bán 5 vé, còn 0 ghế, 5 khách nhận thông báo hết vé. Biểu đồ (Hình 3.29) cho thấy các giao tác xếp hàng: đoạn đỏ rất dài vì mỗi khách giữ khóa suốt thời gian cân nhắc. Bảng khóa lấy mẫu từ performance_schema (Hình 3.30) cho thấy nhiều khách cùng chờ khóa X, REC_NOT_GAP trên bản ghi khóa chính 1 của bảng chuyen_bay. Tổng thời gian chờ khóa là 3.929 ms.



Hình 3.29. Diễn biến khi dùng khóa bi quan: các giao tác xếp hàng

Các khóa InnoDB đã đặt

Lấy mẫu mỗi 25 ms từ `performance_schema.data_locks` trong lúc kịch bản chạy.

MỐC	GIAO TÁC	BẢNG	CHỈ MỤC	LOẠI	CHẾ ĐỘ KHÓA	TRẠNG THÁI	DỮ LIỆU	Ý NGHĨA
2 ms	Khách 5	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)
33 ms	Khách 4	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)
33 ms	Khách 1	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)
33 ms	Khách 10	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)
33 ms	Khách 7	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)
33 ms	Khách 8	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)
33 ms	Khách 3	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)
33 ms	Khách 9	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)
33 ms	Khách 2	chuyen_bay	PRIMARY	RECORD	X_REC_NOT_GAP	WAITING	1	Khóa ghi trên đúng một bản ghi (record lock)

Hình 3.30. Các khóa InnoDB lấy mẫu trong lúc chạy khóa bi quan

Với khóa **lạc quan**, dữ liệu cũng đúng (5 vé) nhưng có tới **35 lần thử lại** do va chạm version (Hình 3.31): mỗi đoạn ngắn trên biểu đồ là một lần thử, kết thúc bằng ROLLBACK. Với **UPDATE nguyên tử**, kết quả đúng và nhanh nhất – 93 ms (Hình 3.32) – vì không có khe hở giữa đọc và ghi, và khóa chỉ được giữ trong thời gian rất ngắn.

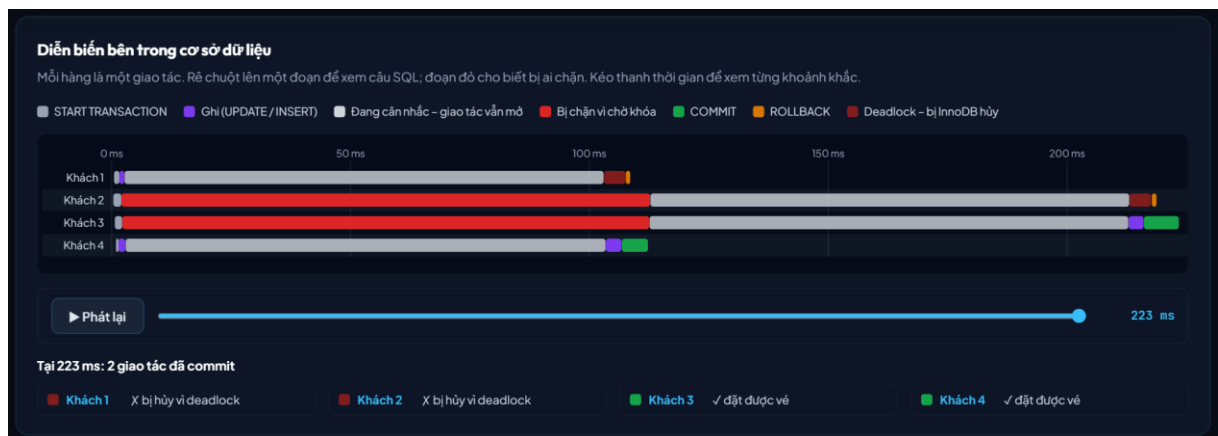


Hình 3.31. Diễn biến khi dùng khóa lạc quan: nhiều lần thử lại



Hình 3.32. Kết quả khi dùng UPDATE nguyên tử

Ở chế độ **deadlock** với 4 khách, InnoDB phát hiện vòng chờ và hủy 2 giao tác (đoạn màu đỏ sẫm trên biểu đồ, Hình 3.33); nhật ký câu lệnh ghi lại lỗi 1213 mà MySQL trả về (Hình 3.34).



Hình 3.33. Diễn biến kịch bản deadlock: InnoDB hủy giao tác nạn nhân

Nhật ký câu lệnh SQL						
Lọc theo hành khách						
Tất cả						
MỐC (MS)	KÉO DÀI	GIAO TÁC	VIỆC	CÂU LỆNH	GHI CHÚ	
0,4	1,7	Khách 2	START TRANSACTION	START TRANSACTION		
0,5	1,1	Khách 1	START TRANSACTION	START TRANSACTION		
0,7	1,5	Khách 3	START TRANSACTION	START TRANSACTION		
0,9	0,5	Khách 4	START TRANSACTION	START TRANSACTION		
1,4	1,4	Khách 4	giữ ghế #2	UPDATE ghe SET trang_thai = 'dang_giu', nguoi_giu = 'Khách 4' WHERE ma_ghe = 2	đặt khóa X lên dòng ghế #2	
1,6	1,0	Khách 1	giữ ghế #1	UPDATE ghe SET trang_thai = 'dang_giu', nguoi_giu = 'Khách 1' WHERE ma_ghe = 1	đặt khóa X lên dòng ghế #1	
2,0	110,7	Khách 2	giữ ghế #2	UPDATE ghe SET trang_thai = 'dang_giu', nguoi_giu = 'Khách 2' WHERE ma_ghe = 2	bị chặn 111 ms vì chờ khóa do Khách 4 đang giữ. đặt khóa X lên dòng ghế #2	

Hình 3.34. Nhật ký câu lệnh SQL của kịch bản deadlock

Bảng 3.7. Kết quả một lần chạy với 10 khách, 5 ghế, cân nhắc 100 ms

Cách xử lý	Vé bán	Bán vượt	Ghế còn	Nhất quán	Thử lại	Chờ khóa	Thời gian
Không dùng khóa	10	5	4	Không	0	8	218 ms
Khóa bí quan	5	0	0	Có	0	9	582 ms
Khóa lạc quan	5	0	0	Có	35	8	725 ms
UPDATE nguyên tử	5	0	0	Có	0	8	93 ms

3.2.9. Chức năng ma trận mức cô lập

Tên chức năng: Ma trận mức cô lập (tab 3). **Chức năng:** chạy 6 thí nghiệm hai phiên ở cả 4 mức cô lập (24 lần chạy) và lập ma trận cho biết hiện tượng nào thực sự xảy ra, đặt cạnh lý thuyết chuẩn SQL (Hình 3.35). Ô viền nét đứt màu cam là chỗ MySQL khác lý thuyết. Bấm vào một ô để xem chi tiết thí nghiệm từng bước, kèm bảng khóa tại lúc có phiên bị chặn.

Ma trận: hiện tượng nào xảy ra ở mức nào?

Kết quả đo thật trên MySQL, đặt cạnh lý thuyết chuẩn SQL. Bấm vào một ô để xem chi tiết.

HIỆN TƯỢNG	READ UNCOMMITTED	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Độc dữ liệu rác (Dirty read)	Xảy ra chuẩn SQL: có	Không chuẩn SQL: không	Không chuẩn SQL: không	Không chuẩn SQL: không có chặn
Độc không lặp lại (Non-repeatable read)	Xảy ra chuẩn SQL: có	Xảy ra chuẩn SQL: có	Không chuẩn SQL: không	Không chuẩn SQL: không có chặn
Bóng ma – đọc thường (Phantom read)	Xảy ra chuẩn SQL: có	Xảy ra chuẩn SQL: có	Không chuẩn SQL: có	Không chuẩn SQL: không có chặn
Bóng ma – đọc có khóa (Gap lock)	Xảy ra chuẩn SQL: có	Xảy ra chuẩn SQL: có	Không chuẩn SQL: có có chặn	Không chuẩn SQL: không có chặn
Mất cập nhật (Lost update)	Xảy ra chuẩn SQL: có	Xảy ra chuẩn SQL: có	Xảy ra chuẩn SQL: không	Không chuẩn SQL: không có chặn - deadlock
Độc nhất quán và đọc hiện hành (MVCC)	Giống nhau	Giống nhau	Khác nhau	Giống nhau có chặn

❌ Lỗi xảy ra
✅ Được ngăn chặn
🔍 MySQL khác lý thuyết chuẩn SQL

Những điểm MySQL/InnoDB khác lý thuyết:

- Bóng ma – đọc thường (Phantom read) - REPEATABLE READ:** Khác chuẩn SQL: InnoDB dùng MVCC nên đọc thường ở REPEATABLE READ không thấy bóng ma.
- Bóng ma – đọc có khóa (Gap lock) - REPEATABLE READ:** Khác chuẩn SQL: InnoDB dùng next-key lock (record + gap) nên chặn được bóng ma cả khi đọc có khóa.
- Mất cập nhật (Lost update) - REPEATABLE READ:** Khác lý thuyết: REPEATABLE READ của MySQL dựa trên snapshot, UPDATE không kiểm tra xung đột ghi nên vẫn mất cập nhật. Phải dùng FOR UPDATE, khóa lạc quan hoặc UPDATE nguyên tử.

Hình 3.35. Ma trận 6 hiện tượng × 4 mức cô lập đo trên MySQL

Bảng 3.8. Kết quả ma trận đo trên MySQL (* = khác lý thuyết chuẩn SQL)

Hiện tượng	READ UNCOMMITTED	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Độc dữ liệu rác	Xảy ra	Không	Không	Không (chặn)
Độc không lặp lại	Xảy ra	Xảy ra	Không	Không (chặn)
Bóng ma (đọc thường)	Xảy ra	Xảy ra	Không *	Không (chặn)
Bóng ma (đọc có khóa)	Xảy ra	Xảy ra	Không * (chặn)	Không (chặn)
Mất cập nhật	Xảy ra	Xảy ra	Xảy ra *	Không (deadlock)

Hiện tượng	READ UNCOMMITTED	READ COMMITTED	REPEATABLE READ	SERIALIZABLE
Độc nhất quán ≠ đọc hiện hành	Giống nhau	Giống nhau	Khác nhau	Giống nhau (chặn)

Ma trận cho thấy ba điểm MySQL/InnoDB khác lý thuyết, đều ở mức REPEATABLE READ: (1) bóng ma khi đọc thường **không** xảy ra vì đọc nhất quán dùng snapshot; (2) bóng ma khi đọc có khóa cũng **không** xảy ra vì next-key lock chặn lệnh chen; (3) mất cập nhật **vẫn xảy ra** vì UPDATE không kiểm tra xung đột ghi. Ở SERIALIZABLE, mất cập nhật được ngăn bằng khóa chết: cả hai giao tác cùng giữ khóa S rồi cùng xin nâng lên khóa X, InnoDB phải hủy một bên. Hàng cuối cho thấy ở REPEATABLE READ, trong cùng một giao tác, SELECT thường và SELECT ... FOR UPDATE có thể trả về hai giá trị khác nhau – lý do khóa bị quan phải dùng FOR UPDATE ngay từ lần đọc đầu tiên.

Hình 3.36 là chi tiết thí nghiệm bóng ma có khóa ở REPEATABLE READ. Bảng khóa tại bước B bị chặn là bằng chứng trực tiếp cho gap lock: A giữ X (next-key) trên các bản ghi (1, 1), (1, 2), (1, 3) của chỉ mục fk_ve_chuyen và X,GAP trên bản ghi (2, 4), còn B chờ khóa X,GAP, INSERT_INTENTION trên đúng khoảng đó – khớp hoàn toàn với Hình 2.4.

Bóng ma – đọc có khóa (Gap lock)

A khóa mọi vé của chuyến VN-808 bằng FOR UPDATE. B cố chèn thêm vé vào chuyến đó. Mức cô lập: REPEATABLE READ

Hiện tượng KHÔNG xảy ra ở mức REPEATABLE READ. A khóa hai lần đầu ra 3 dòng. Lệnh INSERT của B bị chặn bởi GAP LOCK: InnoDB khóa cả khoảng trống sau dòng cuối cùng thỏa ma_chuyen = 1 (next-key lock), nên B phải chờ tới khi A commit. Xem bảng khóa: B chờ khóa INSERT_INTENTION, xung đột với khóa GAP của A.

Theo lý thuyết (chuẩn SQL): có thể xảy ra. MySQL thực tế: không xảy ra. **KHÁC lý thuyết**

Khác chuẩn SQL: InnoDB dùng next-key lock (record + gap) nên chặn được bóng ma cả khi đọc có khóa.

► Dữ liệu ban đầu

◀ Lùi Bước tiếp ▶ Hiện tất cả Bước 9 / 9

#	PHIÊN A	PHIÊN B								
1	A1: A mở giao tác START TRANSACTION OK									
2	A2: A khóa toàn bộ vé của chuyến VN-808 SELECT ma_ve, hanh_khach FROM ve WHERE ma_chuyen = 1 FOR UPDATE <table border="1"> <thead> <tr> <th>MA_VE</th> <th>HANH_KHACH</th> </tr> </thead> <tbody> <tr> <td>1</td> <td>An</td> </tr> <tr> <td>2</td> <td>Bình</td> </tr> <tr> <td>3</td> <td>Chi</td> </tr> </tbody> </table>	MA_VE	HANH_KHACH	1	An	2	Bình	3	Chi	
MA_VE	HANH_KHACH									
1	An									
2	Bình									
3	Chi									
3		B1: B mở giao tác START TRANSACTION OK								
4		BỊ CHẶN B2: B chèn vé mới vào chuyến VN-808 INSERT INTO ve (ma_chuyen, hanh_khach) VALUES (1, 'Hoa') Câu lệnh chưa trả về sau 700 ms - phiên B đang chờ khóa do phiên A giữ.								

▼ Bảng khóa của InnoDB ngay lúc này (12 khóa)

PHIÊN	BẢNG	CHỈ MỤC	LOẠI	CHẾ ĐỘ KHÓA	TRANG THÁI	DỮ LIỆU	Ý NGHĨA
A	ve	—	TABLE	IX	GRANTED	—	Ý định ghi ở mức bảng - bảo trước sẽ khóa X một số dòng
A	ve	PRIMARY	RECORD	X, REC_NOT_GAP	GRANTED	1	Khóa ghi trên đúng một bản ghi (record lock)
A	ve	PRIMARY	RECORD	X, REC_NOT_GAP	GRANTED	2	Khóa ghi trên đúng một bản ghi (record lock)
A	ve	PRIMARY	RECORD	X, REC_NOT_GAP	GRANTED	3	Khóa ghi trên đúng một bản ghi (record lock)
A	ve	fk_ve_chuyen	RECORD	X	GRANTED	1,1	Next-key lock: khóa ghi bản ghi + khoảng trống ngay trước nó
A	ve	fk_ve_chuyen	RECORD	X	GRANTED	1,2	Next-key lock: khóa ghi bản ghi + khoảng trống ngay trước nó
A	ve	fk_ve_chuyen	RECORD	X	GRANTED	1,3	Next-key lock: khóa ghi bản ghi + khoảng trống ngay trước nó
A	ve	fk_ve_chuyen	RECORD	X, GAP	GRANTED	2,4	Gap lock: chỉ khóa khoảng trống, không cho chèn vào
B	chuyen_bay	—	TABLE	IS	GRANTED	—	Ý định đọc có khóa ở mức bảng - bảo trước sẽ khóa S một số dòng
B	chuyen_bay	PRIMARY	RECORD	S, REC_NOT_GAP	GRANTED	1	Khóa đọc trên đúng một bản ghi (record lock)
B	ve	—	TABLE	IX	GRANTED	—	Ý định ghi ở mức bảng - bảo trước sẽ khóa X một số dòng
B	ve	fk_ve_chuyen	RECORD	X, GAP, INSERT_INTENTION	WAITING	2,4	Xin chèn vào một khoảng trống (insert intention)

Hình 3.36. Chi tiết thí nghiệm bóng ma có khóa ở REPEATABLE READ: B bị gap lock chặn

3.2.10. Chức năng so sánh hiệu năng

Tên chức năng: So sánh hiệu năng (tab 4). **Chức năng:** chạy cả bốn cách xử lý trong cùng điều kiện, lặp lại nhiều lần rồi lấy trung bình (Hình 3.37); phần khảo sát tăng dần số

khách để thấy chi phí của từng cách khi tranh chấp tăng (Hình 3.38). Kết quả với 10 khách, 5 ghế, cân nhắc 80 ms, lặp 3 lần được tổng hợp ở Bảng 3.9.



Hình 3.37. So sánh bốn cách xử lý tranh chấp

Bảng 3.9. Kết quả trung bình của bốn cách xử lý

Cách xử lý	Thời gian TB	Nhanh – chậm nhất	Vé bán TB	Bán vượt TB	Lần cho kết quả sai	Thử lại TB	Chờ khóa TB
Không dùng khóa	177 ms	171 – 181 ms	10,0	5,0	3 / 3	0,0	6,3
Khóa bi quan	486 ms	475 – 506 ms	5,0	0,0	0 / 3	0,0	9,0
Khóa lạc quan	655 ms	644 – 662 ms	5,0	0,0	0 / 3	35,0	4,0
UPDATE nguyên tử	92 ms	69 – 133 ms	5,0	0,0	0 / 3	0,0	7,3



Hình 3.38. Khảo sát chịu tải khi số khách tăng dần

Bảng 3.10. Khảo sát chịu tải (cân nhắc 50 ms; số ghế bằng một nửa số khách)

Số khách	Số ghế	Không khóa	Khóa bi quan	Khóa lạc quan	UPDATE nguyên tử
5	2	114 ms vượt 3	123 ms chờ 4	166 ms thử lại 7	28 ms
10	5	124 ms vượt 5	320 ms chờ 9	477 ms thử lại 35	73 ms
20	10	212 ms vượt 10	617 ms chờ 19	908 ms thử lại 109, bỏ cuộc 10	132 ms
40	20	638 ms vượt 20	1.254 ms chờ 39	986 ms thử lại 241, bỏ cuộc 27	232 ms

Từ số liệu có thể rút ra: (1) không dùng khóa nhanh nhưng **sai ở mọi lần chạy** (3/3), bán vượt trung bình 5,0 vé; (2) trong các cách cho kết quả đúng, **UPDATE nguyên tử nhanh nhất** (92 ms) và nên là lựa chọn mặc định khi điều kiện viết được trong một câu lệnh; (3) khóa bi quan (486 ms) chậm vì các giao tác phải xếp hàng suốt thời gian cân nhắc; (4) khóa lạc quan (655 ms) không phải chờ khóa lâu nhưng phải thử lại trung bình 35,0 lần, và khi tranh chấp tăng lên 40 khách thì số lần thử lại lên tới 241, có 27 khách bỏ cuộc. Khóa lạc quan chỉ phù hợp khi xung đột hiếm.

3.3. Kiểm thử và triển khai

Vì thứ cần kiểm chứng chính là hành vi khóa của InnoDB, bộ kiểm thử **không dùng dữ liệu giả lập** mà chạy trực tiếp trên MySQL trong container. Bộ kiểm thử gồm 151 trường hợp chia thành bốn nhóm (Bảng 3.11) và đã được chạy lặp lại nhiều lần để loại trừ kiểm thử chậm chèn. Lần chạy gần nhất cho kết quả **151/151 đạt** trong khoảng 45 giây.

Bảng 3.11. Các nhóm kiểm thử tự động

Tệp kiểm thử	Số ca	Nội dung kiểm tra
tests/test_xungdot.py	56	10 kịch bản sai/đúng cho đúng kết quả; mỗi bước có ảnh chụp dữ liệu và khóa; dòng đang sửa dữ liệu luôn có đúng một chủ khóa X; chủ khóa đúng sau khóa chết.
tests/test_phongthinghiem.py	37	24 ô của ma trận mức cô lập; bất biến của bộ điều phối hai phiên; gap lock xuất hiện ở REPEATABLE READ.
tests/test_kichban.py	39	Tính đúng đắn của từng cách đặt vé ở nhiều quy mô (1–60 khách); tính loại trừ của khóa X; hồi quy lỗi bộ đệm innodb_trx.
tests/test_api.py	19	Kiểm tra tham số, mã lỗi, chặn hai lượt chạy song song (mã 409).

Về triển khai, toàn bộ hệ thống được đóng gói bằng Docker Compose gồm hai dịch vụ mysql (image mysql:9.6, cổng 3310) và app (Python 3.13, cổng 8010). Mã nguồn được chia sẻ trên GitHub tại <https://github.com/thanhvuaws-jpg/hequantricosodemodongthoi>. Thành viên nhóm chỉ cần cài Docker Desktop, tải mã nguồn về và nhấp đúp các tệp sau:

- `khoi-dong.bat` – tự mở Docker Desktop nếu chưa chạy, khởi động MySQL và ứng dụng, chờ sẵn sàng rồi mở trình duyệt tại `http://localhost:8010`.
- `chay-test.bat` – khởi động hệ thống (nếu cần) và chạy toàn bộ 151 kiểm thử, báo ĐẠT hoặc chỉ ra ca lỗi.
- `dung.bat` – tắt hệ thống, cho MySQL tối đa 60 giây để ghi hết dữ liệu xuống đĩa; dữ liệu được giữ lại cho lần sau.

Trong quá trình kiểm tra toàn bộ giao diện để chụp ảnh cho báo cáo, nhóm phát hiện và sửa thêm một số lỗi hiển thị: ô ma trận mức cô lập bị dồn vào một cột do một quy tắc CSS dùng chung tên lớp; bảng khóa bị bóp méo trong một cột hẹp; cột khóa trong khung

CSDL bị cắt ở bảng nhiều cột; nhãn trên biểu đồ diễn biến khó đọc trên nền tối. Tất cả đã được sửa và đưa lên kho mã nguồn.

3.4. Đánh giá và những phát hiện về MySQL/InnoDB

Qua thực nghiệm, nhóm rút ra những điểm đáng chú ý sau – nhiều điểm khác với lý thuyết chuẩn SQL hoặc ít được đề cập trong giáo trình:

- **REPEATABLE READ của MySQL chặn được bóng ma** (chuẩn SQL không yêu cầu), nhờ MVCC cho đọc thường và next-key lock cho đọc có khóa.
- **REPEATABLE READ của MySQL không chặn được mất cập nhật.** UPDATE luôn ghi lên bản mới nhất mà không kiểm tra dòng đã bị sửa sau snapshot. Tệ hơn, khi giá trị ghi trùng giá trị cũ, MySQL chỉ báo Rows matched: 1, Changed: 0, ứng dụng không có cách nào biết dữ liệu vừa bị mất.
- **Ở SERIALIZABLE, mất cập nhật được ngăn bằng khóa chết:** hai bên cùng giữ khóa S rồi cùng xin nâng lên khóa X; ứng dụng phải sẵn sàng bắt lỗi 1213 và chạy lại.
- **Gap lock quan sát được tận mắt:** ở REPEATABLE READ, A giữ X, GAP trên bản ghi chỉ mục (2, 4), B chờ X, GAP, INSERT_INTENTION trên đúng khoảng đó; ở READ COMMITTED không có gap lock nên B chèn được.
- **INSERT không đơn nguyên ở mức vật lý:** khi INSERT của B bị chặn, dòng mới đã nằm trong chỉ mục chính (đọc ở READ UNCOMMITTED đã thấy), chỉ đang kẹt ở chỉ mục phụ.
- **Dòng mới chèn chỉ có khóa ngầm** (implicit lock): mã giao tác được ghi ngay trong dòng, data_locks không liệt kê cho tới khi có giao tác khác đụng vào.
- **Ở REPEATABLE READ, UPDATE giữ khóa cả trên dòng không khớp điều kiện:** trong kịch bản khóa chết đúng, B vẫn giữ khóa X trên ghế 12A dù Rows matched: 0.

Đối chiếu với mục tiêu ở Chương 1, ứng dụng đã tái hiện đủ năm xung đột với kịch bản sai/đúng, đo đạc được bốn cách xử lý tranh chấp, lập được ma trận mức cô lập và chỉ ra chính xác những điểm MySQL khác lý thuyết; mọi kết quả đều kiểm chứng được bằng bộ kiểm thử tự động.

KẾT LUẬN

Kết quả đạt được. Đề tài đã hệ thống hóa lý thuyết điều khiển truy xuất đồng thời – giao tác, lịch giao tác, các hiện tượng bất thường, kỹ thuật khóa, giao thức khóa hai pha, MVCC, mức cô lập – và làm rõ cách MySQL/InnoDB cài đặt các kỹ thuật đó. So với mục tiêu đề ra, nhóm đã xây dựng hoàn chỉnh ứng dụng minh họa với bốn chức năng: năm cặp kịch bản xung đột sai/đúng có hoạt hình lịch giao tác và bảng khóa thật; mô phỏng tới 60 khách đặt vé đồng thời với bốn cách xử lý; ma trận 24 thí nghiệm mức cô lập; so sánh và khảo sát hiệu năng. Hệ thống chạy được bằng một cú nhấp nhờ Docker và được bảo đảm bởi 151 kiểm thử tự động trên MySQL thật.

So với các tài liệu chỉ trình bày bảng mức cô lập theo lý thuyết, đóng góp nổi bật của đề tài là **kiểm chứng bằng thực nghiệm** và chỉ ra những điểm MySQL khác chuẩn SQL: REPEATABLE READ chặn được bóng ma nhưng không chặn được mất cập nhật, SERIALIZABLE ngăn mất cập nhật bằng khóa chết, cùng các chi tiết về gap lock, khóa ngầm và những bẫy khi giám sát khóa qua performance_schema.

Bài học kinh nghiệm. Không nên tin rằng “có giao tác là dữ liệu đúng”: cần chọn kỹ thuật phù hợp với từng nghiệp vụ. Khi điều kiện viết được trong một câu lệnh, UPDATE nguyên tử là lựa chọn đúng và nhanh nhất; khi phải đọc rồi mới quyết định, cần khóa bi quan bằng FOR UPDATE ngay từ lần đọc đầu; khóa lạc quan chỉ phù hợp khi tranh chấp hiếm. Mọi giao tác nên xin khóa theo một thứ tự thống nhất, và ứng dụng luôn phải sẵn sàng xử lý lỗi 1213, 1205 bằng cách chạy lại giao tác. Về mặt kỹ thuật, nhóm cũng học được cách điều phối nhiều kết nối song song và cách đọc đúng thông tin khóa của InnoDB.

Hạn chế. Đề tài mới khảo sát MySQL trên một máy chủ đơn; chưa xét các hiện tượng phức tạp hơn như lệch ghi (write skew), chưa so sánh với hệ quản trị khác và chưa đo hiệu năng ở quy mô lớn.

Hướng phát triển. Trước mắt, có thể bổ sung kịch bản lệch ghi và kịch bản thứ tự khóa trên nhiều bảng, cho phép người dùng tự soạn kịch bản hai phiên ngay trên giao diện, và xuất lịch giao tác ra tệp để đưa vào bài giảng. Về lâu dài, ứng dụng có thể mở rộng sang PostgreSQL (vốn phát hiện xung đột ghi ở REPEATABLE READ) để so sánh trực tiếp hai

cách cài đặt mức cô lập, khảo sát hành vi đồng thời trên hệ thống nhân bản, và tích hợp vào bài giảng thực hành của học phần.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [3] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil, and P. O'Neil, “A critique of ANSI SQL isolation levels,” in *Proc. ACM SIGMOD Int. Conf. Management of Data*, San Jose, CA, USA, 1995, pp. 1–10. DOI: 10.1145/223784.223785.
- [4] Oracle Corporation, “InnoDB Locking,” *MySQL Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/en/innodb-locking.html> (truy cập 10/2026).
- [5] ISO/IEC 9075-2:2016, *Information technology — Database languages — SQL — Part 2: Foundation (SQL/Foundation)*. Geneva, Switzerland: ISO, 2016.
- [6] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. San Francisco, CA, USA: Morgan Kaufmann, 1993.
- [7] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Reading, MA, USA: Addison-Wesley, 1987.
- [8] Oracle Corporation, “Consistent Nonlocking Reads” và “Locking Reads,” *MySQL Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/en/innodb-consistent-read.html> (truy cập 10/2026).
- [9] Oracle Corporation, “InnoDB Multi-Versioning,” *MySQL Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/en/innodb-multi-versioning.html> (truy cập 10/2026).
- [10] Oracle Corporation, “Deadlocks in InnoDB,” *MySQL Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/en/innodb-deadlocks.html> (truy cập 10/2026).
- [11] Oracle Corporation, “The data_locks Table,” *MySQL Reference Manual*. [Online]. Available: <https://dev.mysql.com/doc/refman/en/performance-schema-data-locks-table.html> (truy cập 10/2026).

- [12] S. Ramírez, “FastAPI Documentation.” [Online]. Available: <https://fastapi.tiangolo.com> (truy cập 10/2026).
- [13] Docker Inc., “Docker Compose Documentation.” [Online]. Available: <https://docs.docker.com/compose/> (truy cập 10/2026).

PHỤ LỤC

Phụ lục A. Hướng dẫn cài đặt và chạy

- Cài Docker Desktop (Windows 10/11) và mở nó lên một lần.
- Tải mã nguồn: `git clone https://github.com/thanhvuaws-jpg/hequantricosodemongthoi.git` (hoặc tải tệp ZIP từ GitHub và giải nén).
- Nhấp đúp `khoi-dong.bat`. Lần đầu cần vài phút để tải image MySQL và Python; các lần sau chỉ vài giây.
- Trình duyệt tự mở `http://localhost:8010`. Muốn chạy kiểm thử: nhấp đúp `chay-test.bat` (không thao tác trên giao diện trong lúc chạy kiểm thử). Muốn tắt: nhấp đúp `dung.bat`.
- Có thể kết nối trực tiếp tới MySQL của demo tại cổng 3310, tài khoản root / demo123, CSDL `demo_dongthoi`.

Phụ lục B. Lược đồ cơ sở dữ liệu (schema.sql)

```
CREATE TABLE chuyen_bay (  
    ma_chuyen    INT PRIMARY KEY,  
    so_hieu      VARCHAR(100) NOT NULL,          -- ví dụ: VN-808  
    tong_ghe     INT          NOT NULL,  
    so_ghe_trong INT          NOT NULL,  
    version      INT          NOT NULL DEFAULT 0  -- phục vụ khóa lạc quan  
) ENGINE=InnoDB;  
  
CREATE TABLE ve (  
    ma_ve        INT AUTO_INCREMENT PRIMARY KEY,  
    ma_chuyen    INT          NOT NULL,  
    hanh_khach   VARCHAR(50)  NOT NULL,  
    thoi_diem    DATETIME(6)  NOT NULL DEFAULT CURRENT_TIMESTAMP(6),  
    CONSTRAINT fk_ve_chuyen FOREIGN KEY (ma_chuyen)  
        REFERENCES chuyen_bay(ma_chuyen)  
) ENGINE=InnoDB;  
  
CREATE TABLE ghe (  
    ma_ghe       INT PRIMARY KEY,  
    ma_chuyen    INT          NOT NULL,  
    so_ghe       VARCHAR(10)  NOT NULL,          -- ví dụ: 12A  
    trang_thai   VARCHAR(20)  NOT NULL DEFAULT 'trong',  
    nguoi_giu    VARCHAR(50)  NULL,  
    CONSTRAINT fk_ghe_chuyen FOREIGN KEY (ma_chuyen)  
        REFERENCES chuyen_bay(ma_chuyen)  
) ENGINE=InnoDB;
```

Phụ lục C. Cấu trúc mã nguồn

```
hequantricosodemodongthoi/
├── docker-compose.yml      # MySQL 9.6 + ứng dụng
├── Dockerfile
├── schema.sql              # tạo CSDL (chạy tự động lần đầu)
├── khai-dong.bat · chay-test.bat · dung.bat
├── app/
│   ├── csdl.py             # kết nối, mức cô lập, khóa "một kịch bản mỗi lúc"
│   ├── kichban.py          # nhiều khách đặt vé, theo dõi khóa, hiệu năng
│   ├── phongthinghiem.py   # bộ điều phối hai phiên A/B + ma trận mức cô lập
│   ├── xungdot.py          # 5 xung đột × 2 kịch bản sai/đúng
│   ├── main.py             # API FastAPI
│   └── static/             # giao diện (HTML, CSS, JS cho từng tab)
├── tests/                  # 151 kiểm thử chạy trên MySQL thật
└── bao-cai/               # báo cáo, công cụ chụp ảnh và vẽ sơ đồ
```